Neural Networks

Data Science and Management

Corso di Laurea Magistrale in Ingegneria Gestionale

Marco Mamei, Natalia Hadjidimitriou

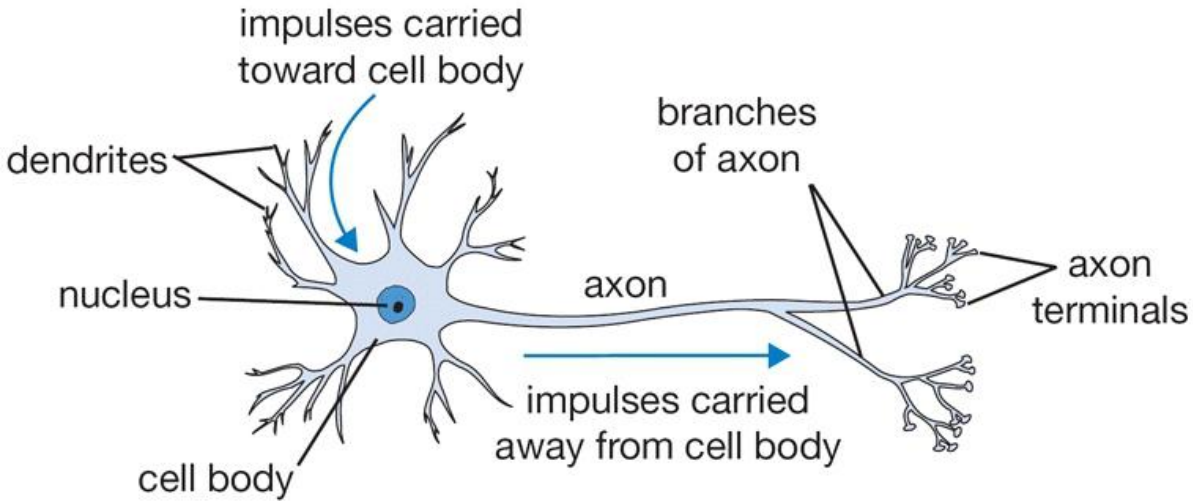
{marco.mamei, selini}@unimore.it

- Neural Networks
- Backpropagation
- Activation Function
- Hyperparameters
- Learning process

Neural Networks

Biologically inspired by the human brain

- Interconnected cells, signal propagation (synapses)

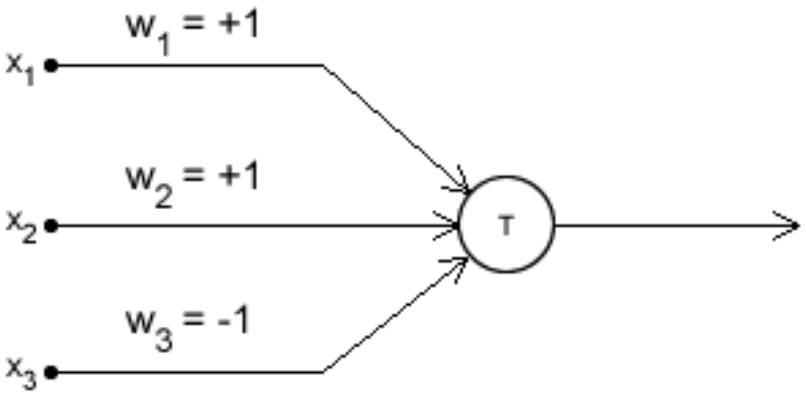


[Image from http://cs231n.github.io/]

Neural Networks

The idea of artificial neuron dates back to 1940s

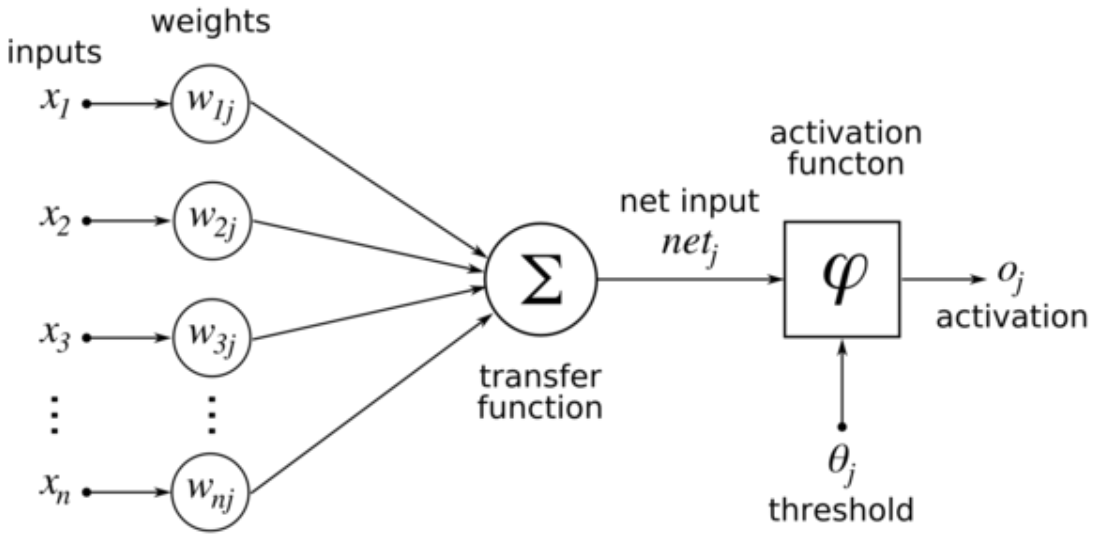
- 1943: McCulloch and Pitts



[Image from http://aishack.in]

Neural Networks

- 1958: Rosenblatt's perceptron



[Image from Wikipedia]

Perceptron

How to learn parameters

- Define a **loss** function to be **minimized**
- Typically the **classification error** on the training set

$$E(\beta, \mathcal{D}) = \sum_{(x_i, y_i) \in \mathcal{D}} \ell(y_i, f(x_i))$$

- How can we **minimize** this error?

Perceptron

For the classic perceptron the **loss function** is defined as:

$$\ell(y_i, f(x_i)) = -y_i f(x_i)$$

Idea: **different signs** between target and prediction imply **wrong classifications**, so we should correct them

Perceptron

Gradient descent algorithm

- Compute the **gradient** of error function wrt parameters
- **Iterate** until gradient is approximately zero

$$\beta^i = \beta^{i-1} - \eta \nabla E(\beta, \mathcal{D})$$

i IS THE ITERATION

- η is called the **learning rate**

Perceptron

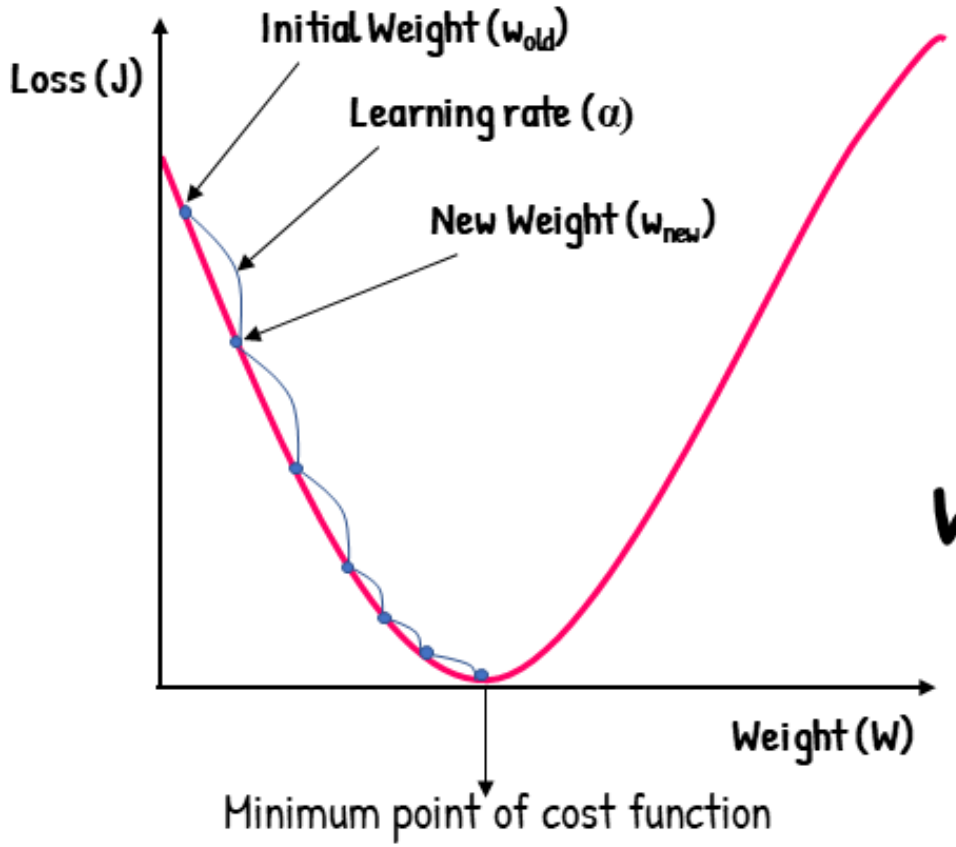
The learning rate is a **crucial** parameter

- Too **small** implies slow **convergence**
- Too **high** causes **instability**
- There exist algorithms to **adapt it during training**

Guarantee to reach a **local minimum** of the error function

Backpropagation

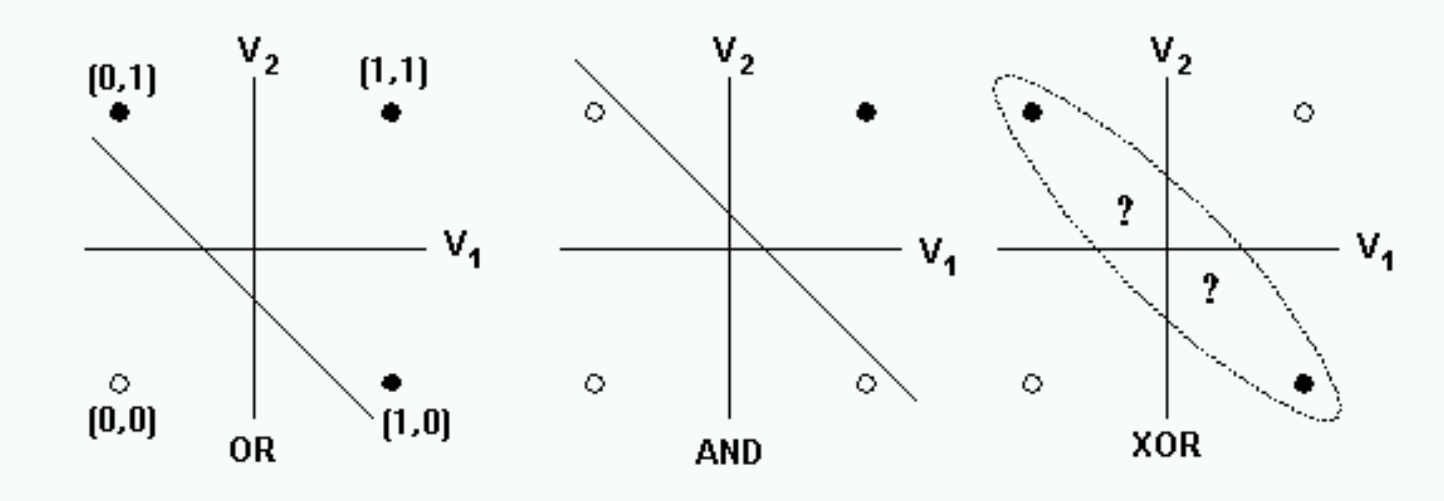
Gradient Descent



$$w_{new} = w_{old} - \alpha \frac{\delta J}{\delta w}$$

Neural Networks

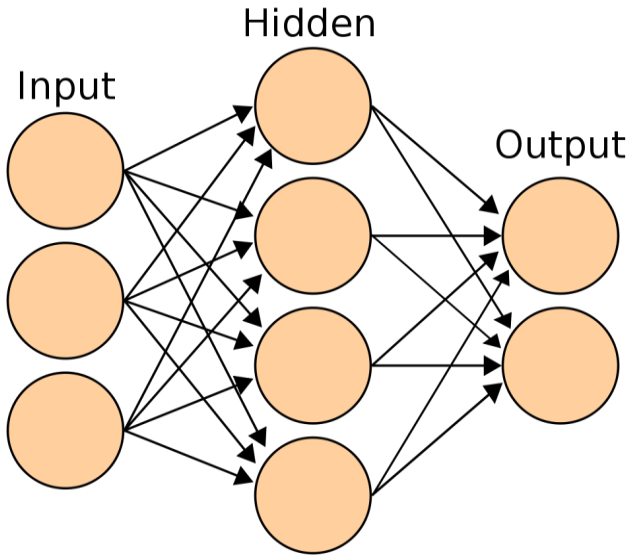
- 1969: Minsky & Papert: limits of the perceptron



[Image from colorado.edu]

Neural Networks

- 1970s-1980s: Multi-Layer Perceptron



[Image from Wikipedia]

Neural Networks

- 1970s-1980s: Multi-Layer Perceptron
  The output of a node is the weighted sum of its inputs, to which an **activation function** is applied:

$$o_j^\ell = \sigma\left(\sum_{i=1}^{m} w_{ij}^\ell o_i^{\ell-1}\right)$$

  The activation function must be non linear, otherwise multilayers are useless

  Weights are learned so as to minimize the error on the output layer (e.g., the squared error)
  This is called the **backpropagation algorithm**

Backpropagation

Backpropagation — Main ideas…

- Initialize the network with **random** weights
- Show the training examples to the network (**epoch**) and **adjust** the weights so that the output matches the desired output (**target**)
- Keep **iterating** over epochs until some **convergence criterion** is met (more on this later)

Backpropagation

Backpropagation — Main ideas…

- The learning phase exploits a **gradient descent** algorithm such as for the perceptron

$$w_{ij}^{t+1} = w_{ij}^{t} - \eta \frac{\partial E}{\partial w_{ij}}$$

BATCH OR ONLINE AS FOR PERCEPTRON
(More on this later…)

Backpropagation - Loss

Loss functions (some examples for classification)

y=1, p(y)=1, correct example, E = 0
y=1, p(y)=0, wrong example, E = +Inf

- Binary cross-entropy

$$E = -\frac{1}{N}\sum_{i=1}^{N} y_i \log(p(y_i)) + (1 - y_i) \log(1 - p(y_i))$$

- Multinomial cross-entropy / categorical cross-entrpy

$$E = -\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{K} y_{ij} \log(p(y_{ij}))$$

Backpropagation - Output

Output nodes

- Binary classification (1 output node)

$$o^L = \sigma(z) = \frac{1}{1 + e^{-z}}$$

SIGMOID
(probability of class 1)

- Multinomial classification (k output nodes)

$$o_i^L = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

SOFTMAX
(probabilities over classes)

Backpropagation - Loss

- Loss functions (some examples for regression)

Residual Sum of Squares (RSS)

$$RSS = \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

where $\hat{y}_i$ is computed as

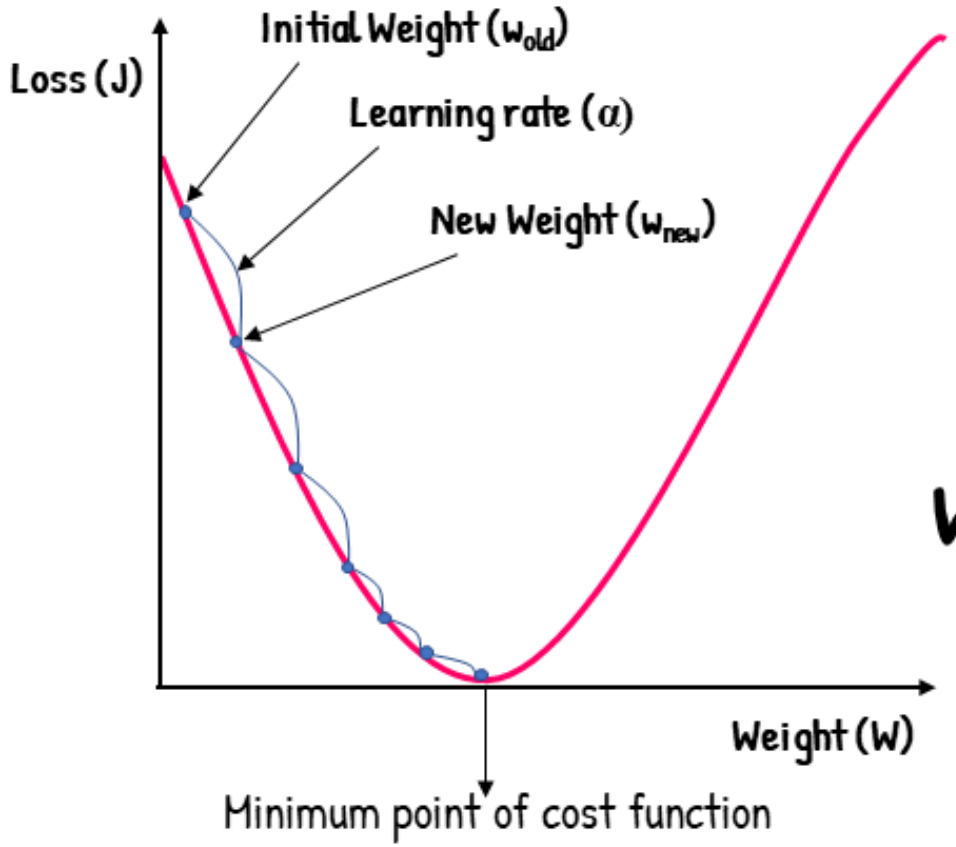
$$o_j^\ell = \sigma\left(\sum_{i=1}^{m} w_{ij}^\ell o_i^{\ell-1}\right)$$

$\hat{y}_i$ = last output

Last output can be sigmoid?

Backpropagation

Gradient Descent



$$w_{new} = w_{old} - \alpha \frac{\delta J}{\delta w}$$

Backpropagation

Backpropagation — High level code…

```
1. init network weights
2. repeat
3.    for each training pair (x,y)
3a.      prediction = compute output(w,x)
3b.      compute error (prediction - y) at the output units
3c.      propagate error from output layer to hidden layer
3d.      propagate error from hidden layer to input layer
3e.      update all weights by gradient step
4. until stopping criterion (more on this later)
```

Backpropagation

Batch vs. **stochastic** gradient descent

- In general, the gradient is obtained as the **sum** of gradients computed for all the **training examples**
- In principle, we could also update the vector of parameters **after each example** rather than after all the training set
- Update after each example is called **stochastic** update
- **Faster** training, which can help **avoiding** local minima
  - Helps avoiding positive and negative example cancelling each other

- **Mini-batch**

Activation functions

Examples of activation functions:

- Sigmoid
- Hyperbolic Tangent (Tanh)
- Rectifier (ReLU — Rectified Linear Unit)
- …

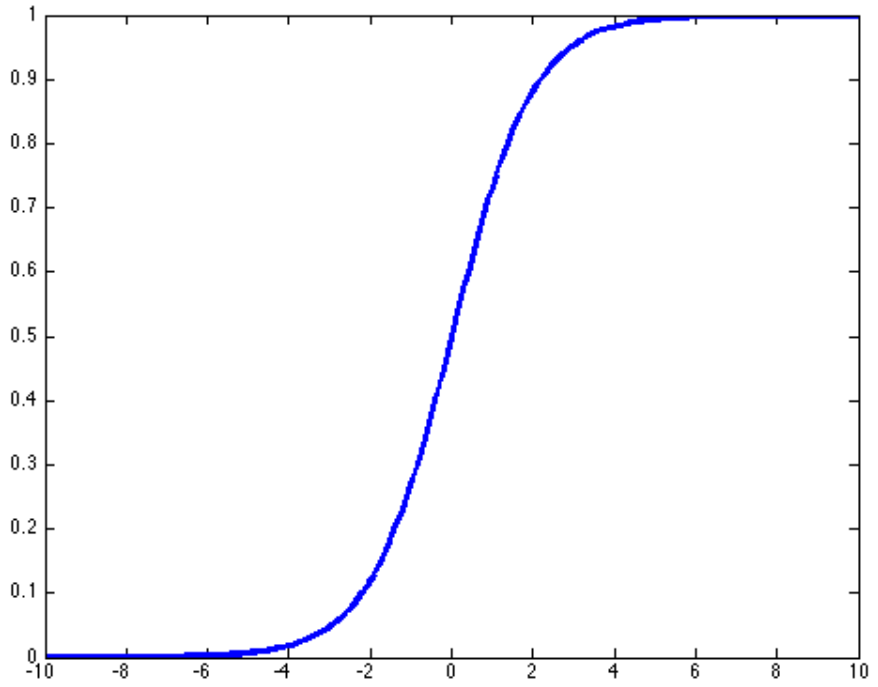
CLASSIC MODELS

DEEP MODELS

Activation functions

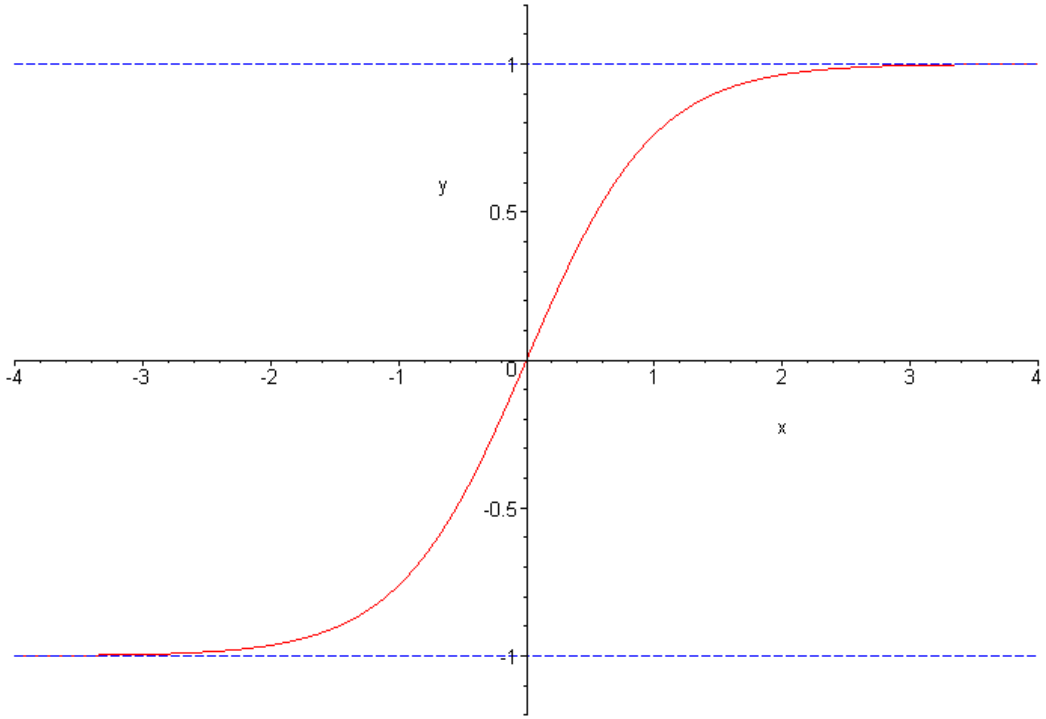
Sigmoid

$$\sigma(t) = \frac{1}{1+e^{-t}}$$



Neural Activation functions

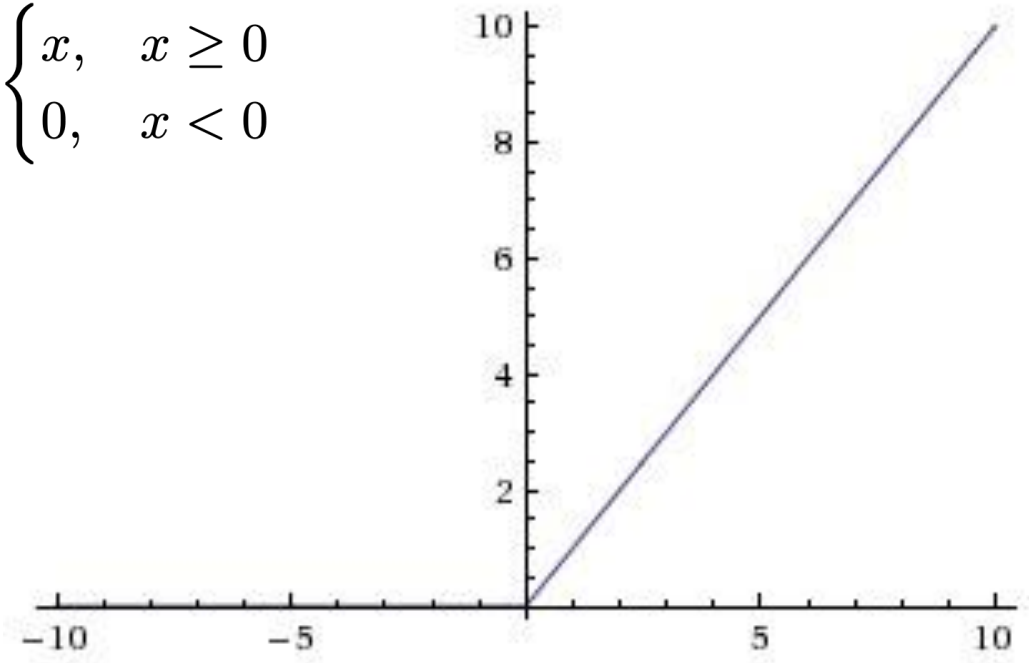
Hyperbolic Tangent



Activation functions

Rectifier

$$ReLU(x) = \begin{cases} x, & x \geq 0 \\ 0, & x < 0 \end{cases}$$



Non-linearity

These activation functions are all **non-linear**

Neural Networks can learn **non-linear dependencies** between input and output, **without any assumption** on the shape of the function

Often, networks with more layers (**deep learning**) are more powerful models (but learning is more costly)

Vanishing gradients

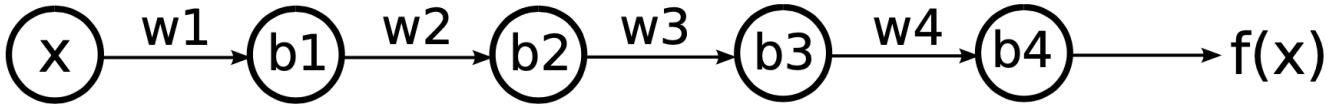
Networks with many (more than 1-2) hidden **layers** and **sigmoid** (a little less with **tanh**…) as activation function suffer of the problem of **vanishing gradients**

Gradients tend to **vanish** (or explode) as we backpropagate errors from the output to the input…

This is mainly due to **composition of derivatives**…

Vanishing gradients

Consider a neural network with 4 layers and **a single unit** in each layer.



We name x the input, w_j and b_j the weight and bias of layer j.
The output of each layer can be computed as:

$$z_j = \sigma(w_j z_{j-1} + b_j)$$

where $z_0 = x$.

[Example by M. Nielsen]

Vanishing gradients

chain rule

$$\frac{dy}{dx} = \frac{dy}{du}\frac{du}{dx}$$

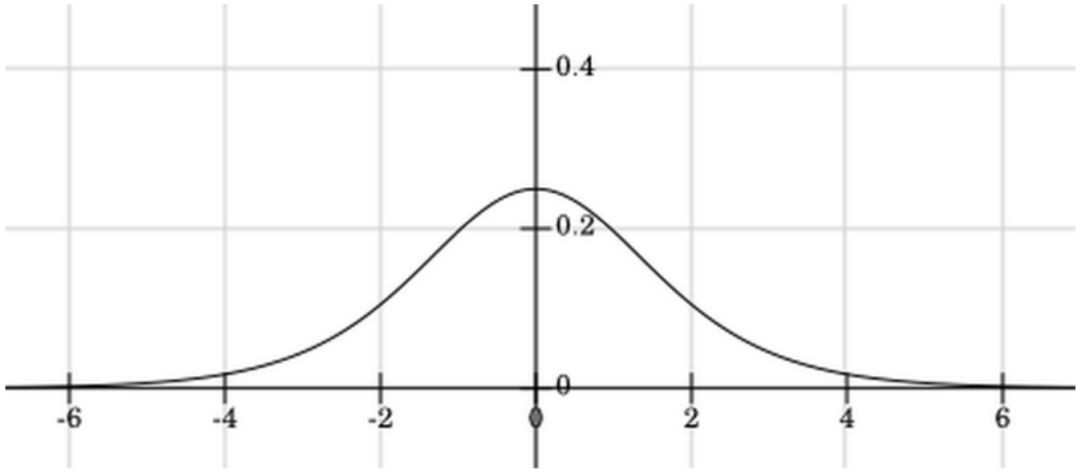
Let us compute the gradient of f with respect to b_1 and b_3:

$$\frac{\partial f}{\partial b_1} = \sigma'(z_1) \cdot w_2 \cdot \sigma'(z_2) \cdot w_3 \cdot \sigma'(z_3) \cdot w_4 \cdot \sigma'(z_4) \cdot \frac{\partial f}{\partial z_4}$$

$$\frac{\partial f}{\partial b_3} = \sigma'(z_3) \cdot w_4 \cdot \sigma'(z_4) \cdot \frac{\partial f}{\partial z_4}$$

Vanishing gradients

If we now consider the derivative of the sigmoid function:



we have that, with typical weight initialization (randomly sampled from a Gaussian with mean 0 and standard deviation 1):

$$|w_j \sigma'(z_j)| < \frac{1}{4}$$

Vanishing gradients

Now going back to the gradient computation:

$$\frac{\partial f}{\partial b_1} = \sigma'(z_1) \cdot w_2 \cdot \sigma'(z_2) \cdot w_3 \cdot \sigma'(z_3) \cdot w_4 \cdot \sigma'(z_4) \cdot \frac{\partial f}{\partial z_4}$$

$$\frac{\partial f}{\partial b_3} = \sigma'(z_3) \cdot w_4 \cdot \sigma'(z_4) \cdot \frac{\partial f}{\partial z_4}$$

Lower layers will tend to have either vanishing or exploding gradients !

Exhaustive analysis for Recurrent Neural Networks in [Hochreiter et el., 2001]

Hyperparameters

When creating a neural network, one has to choose:

- the number of **layers**
- the number of **hidden units** in each layer
- the **number of epochs**
- the **learning rate**…?

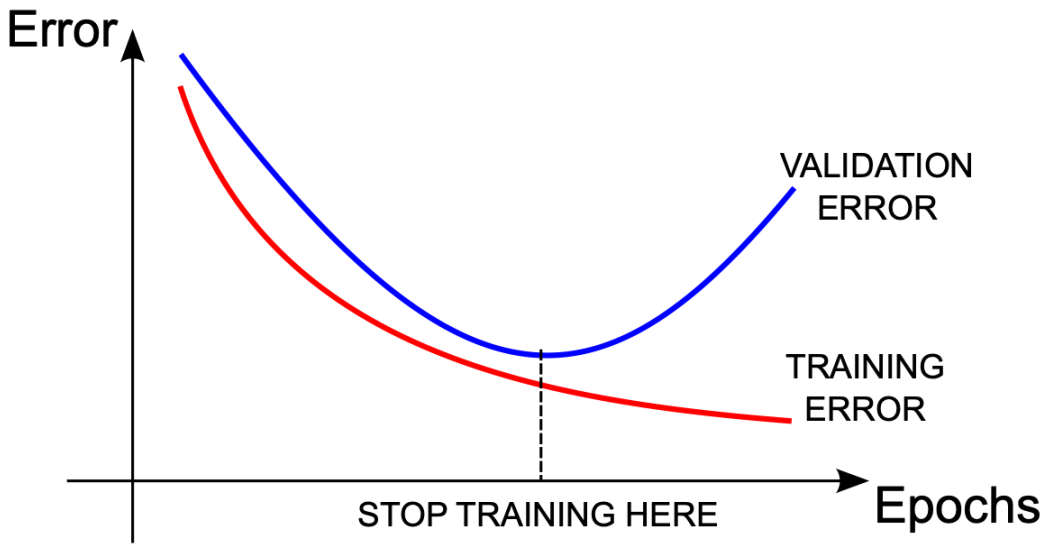
Hyperparameters optimization via validation set!

Hyperparameters

- The number of **hidden layers** has long been 1-2, but now much deeper networks are employed (but in that case **use ReLU activation**)
- The number of **hidden units** in each layer is a set of hyper-parameters that should be chosen according to the **validation** error

Overfitting

Given a network with fixed architecture, the **number of epochs** is chosen following the **early stopping** principle, using a **validation set**



Overfitting

One can also wait for a few epochs to see whether the validation error starts decreasing again (**patience**)

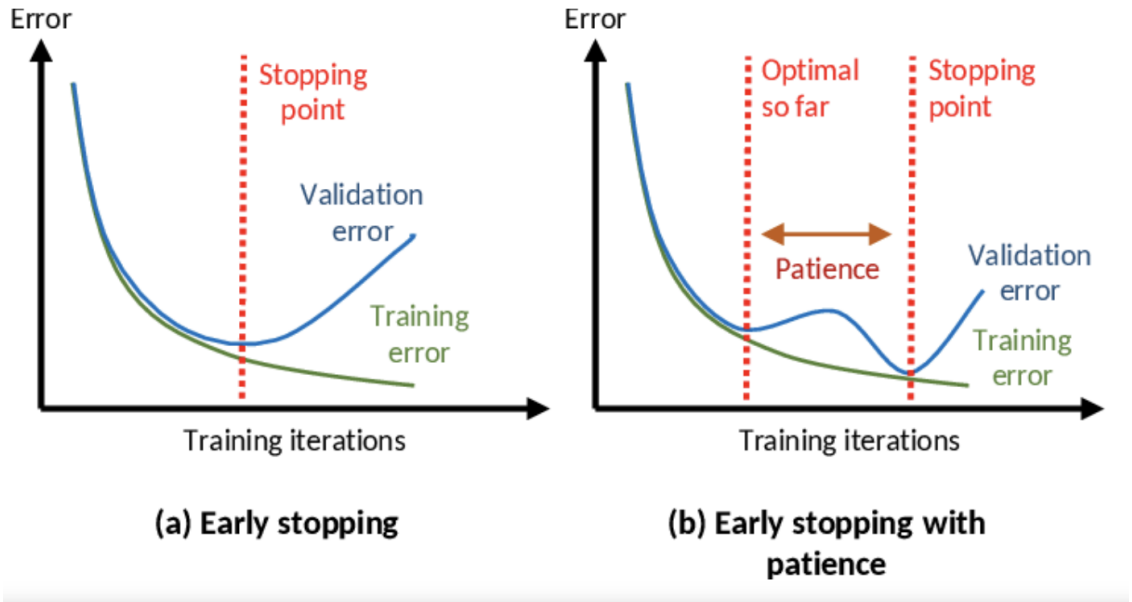


Figure by [Lore et al., 2016]

Learning process

Choosing the learning rate is **tricky**

- Too small: slow convergence
- Too large: divergence (numerically unstable)

This simple method is **obsolete**, we have nowadays more sophisticated optimizers that drive learning

The key idea is to **adjust** the learning rate across the epochs, and have a different one for **each** parameter

Learning process

Stochastic Gradient Descent

$$w_{ij}^{t+1} = w_{ij}^{t} - \eta \frac{\partial E}{\partial w_{ij}}$$

Momentum

$$w_{ij}^{t+1} = w_{ij}^{t} - \eta \frac{\partial E}{\partial w_{ij}} - \mu \left(\frac{\partial E}{\partial w_{ij}} \right)^{t-1}$$

Learning process

Adam

Key idea: one learning rate for each parameter!

INITIAL LEARNING RATE

GRADIENT

$$w_{t+1} = w_t + \Delta w_t$$

$$\Delta w_t = -\eta \frac{\nu_t}{\sqrt{s_t + \epsilon}} g_t$$

$$\nu_t = \beta_1 \nu_{t-1} - (1 - \beta_1) g_t$$

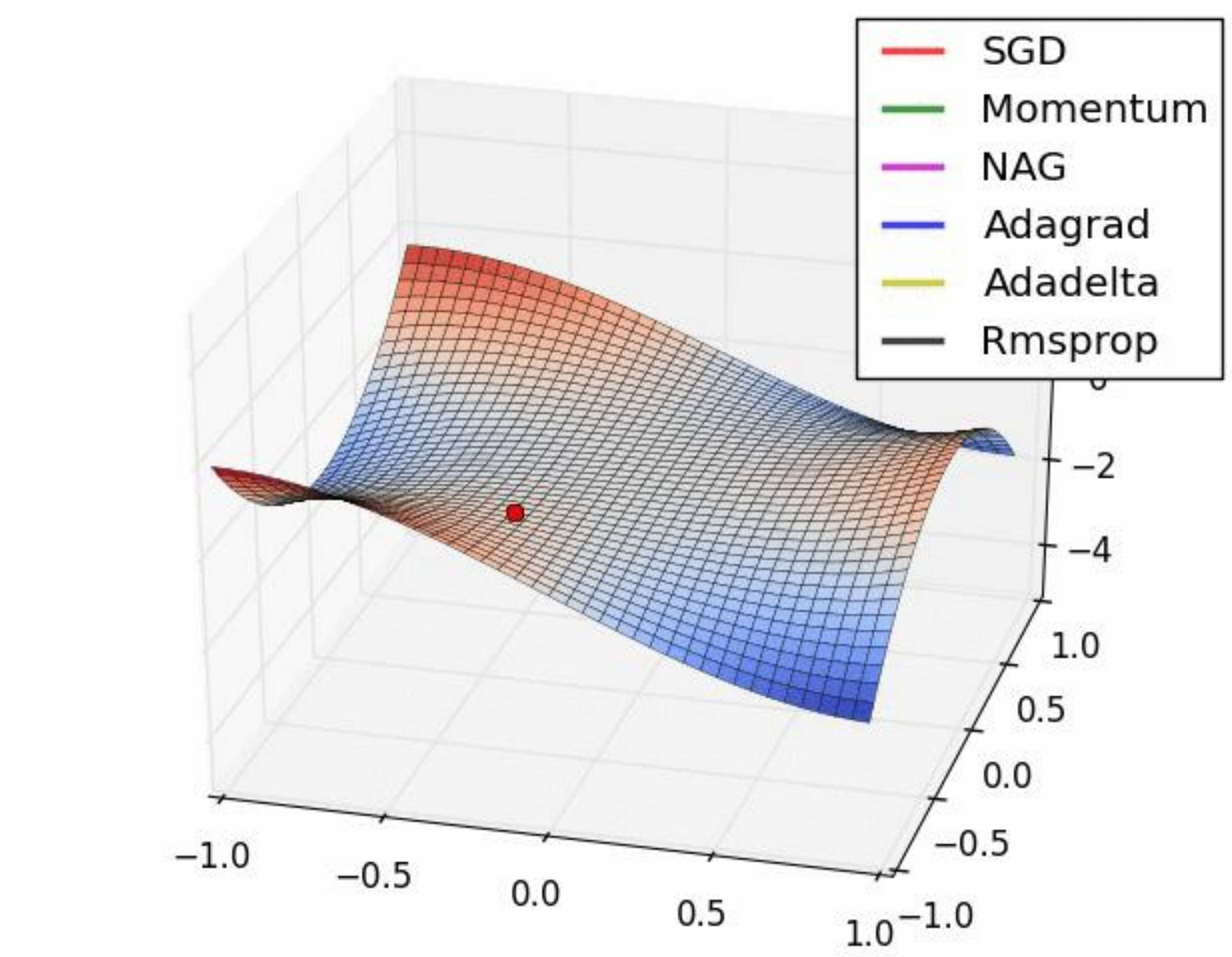
EXPONENTIAL AVERAGE OF SQUARES OF GRADIENTS

$$s_t = \beta_2 s_{t-1} - (1 - \beta_2) g_t^2$$

THIS IS THE MOST WIDELY EMPLOYED OPTIMIZER NOWADAYS

https://towardsdatascience.com/a-visual-explanation-of-gradient-descent-methods-momentum-adagrad-rmsprop-adam-f898b102325c

SGd vs Momentum vs. Rmsprop



Neural Networks

Some tricks of the trade

- One-hot encoding
- Input range and standardization (zero-mean input)
- Smart weight initialization

Many packages for neural network with Python

- Scikit-learn (limited)
- Keras
- Tensorflow
- PyTorch

Deep Learning

- Intro
- Convolutional Networks
- Autoencoders
- RNN - LSTM

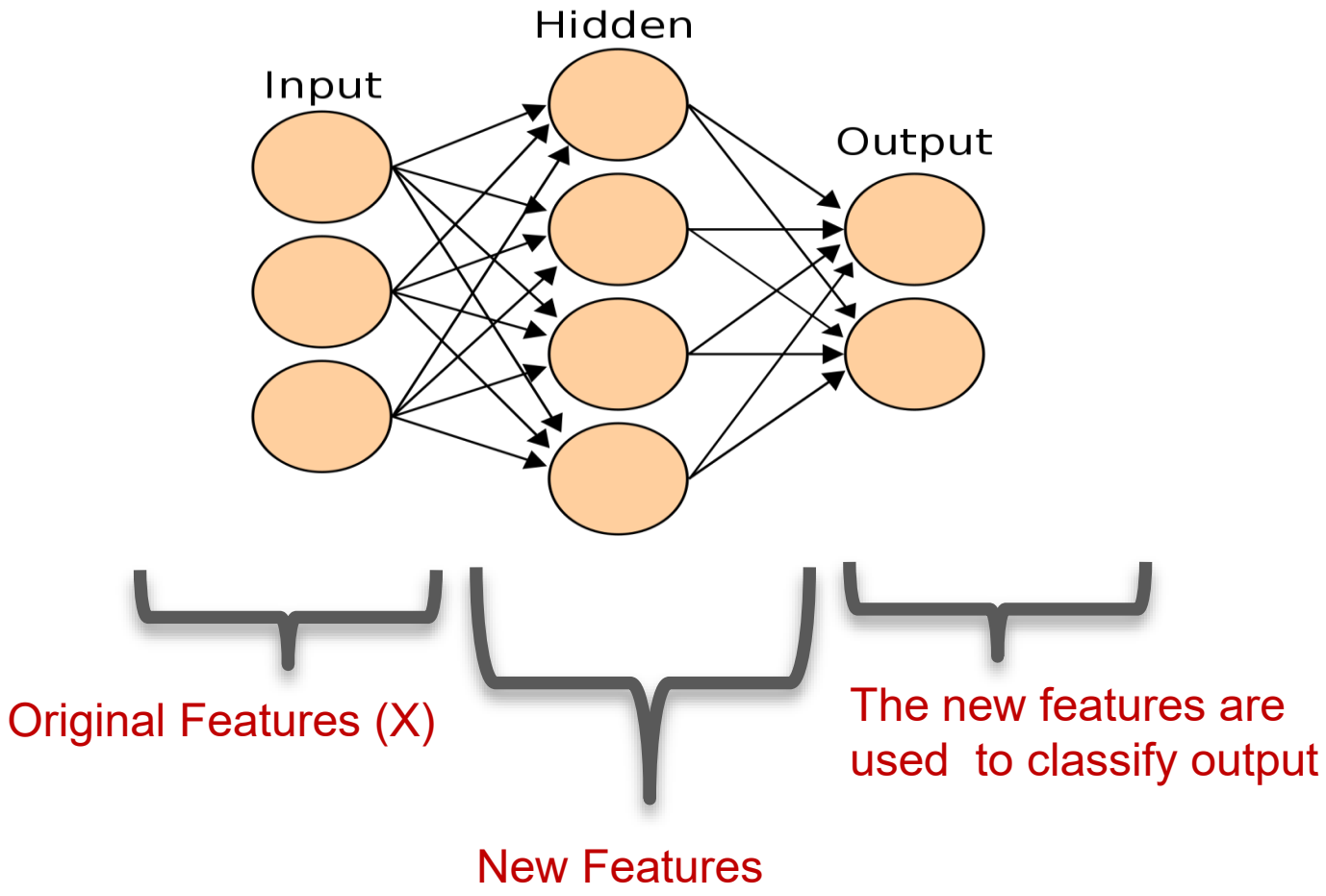
Deep Learning

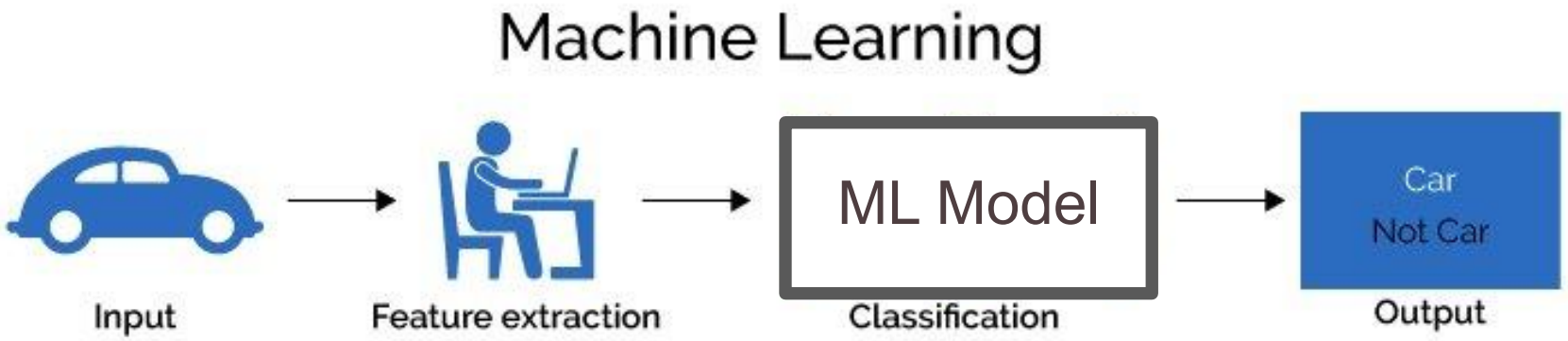
Deep learning is a **hot topic** in artificial intelligence and probably in computer science in general

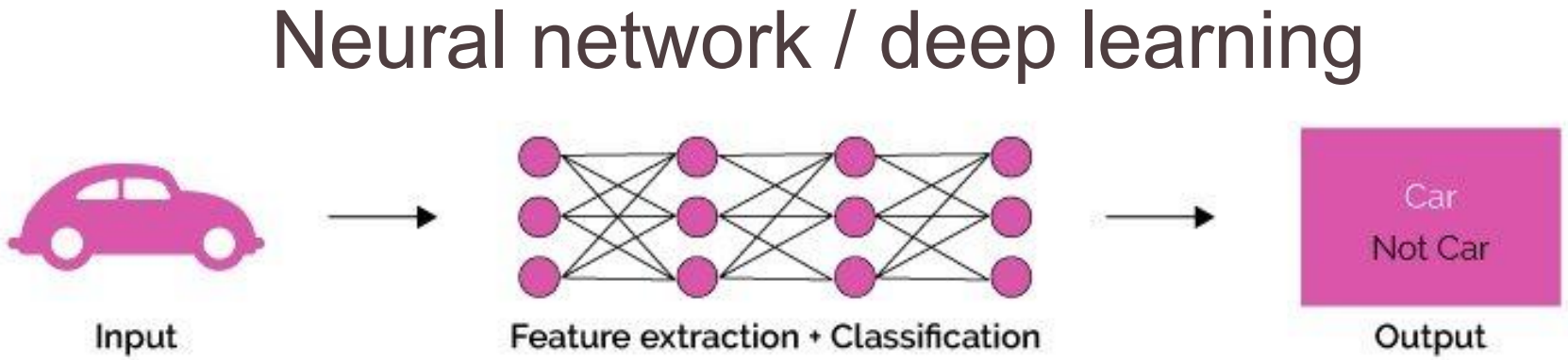
It does **not** mean just “neural networks with many layers” but it includes a wide collection of **architectures (i.e. neurons’ configuration)** useful for many purposes.

- Convolutional neural networks
- Recurrent neural networks
- Recursive neural networks
- Attention-based networks
- Transformers
- …

Feature Creation




Machine Learning
ML Model
Car
Not Car
Input
Feature extraction
Classification
Output


Neural network / deep learning
Car
Not Car
Input
Feature extraction + Classification
Output

Convolutional Neural Networks

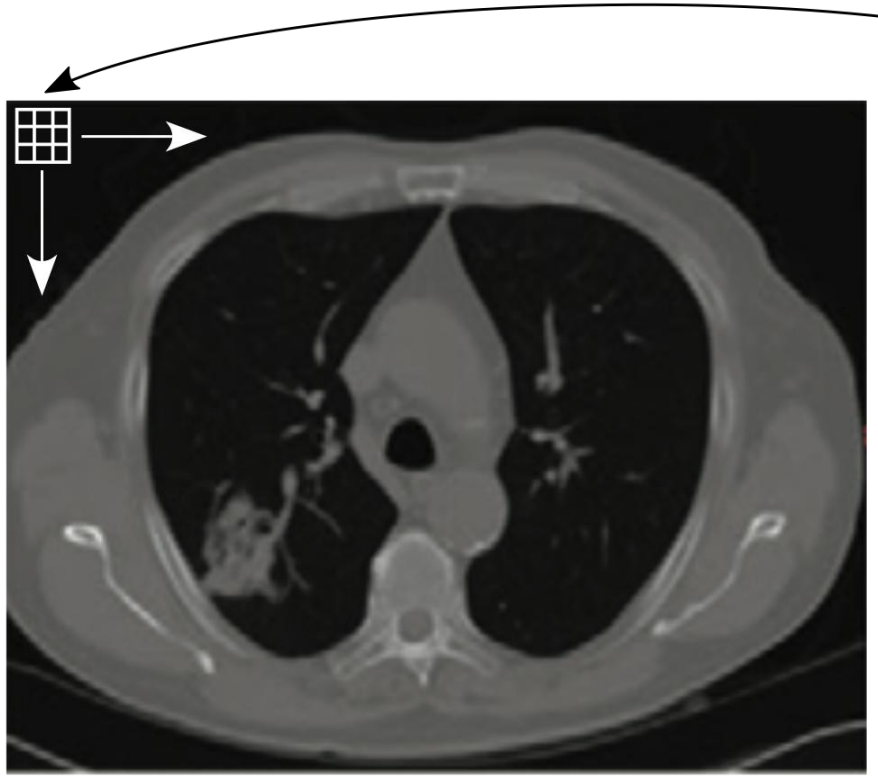
Convolutional Neural Networks

One of the most **widely** used deep network models

- First idea **dates back** to the 80s-90s!!!
- Initially designed to classify **images**
- Later on extended to **other** inputs (e.g., text)

- I could also apply MLP to images, but I'd lose geometry information

Convolutional Neural Networks

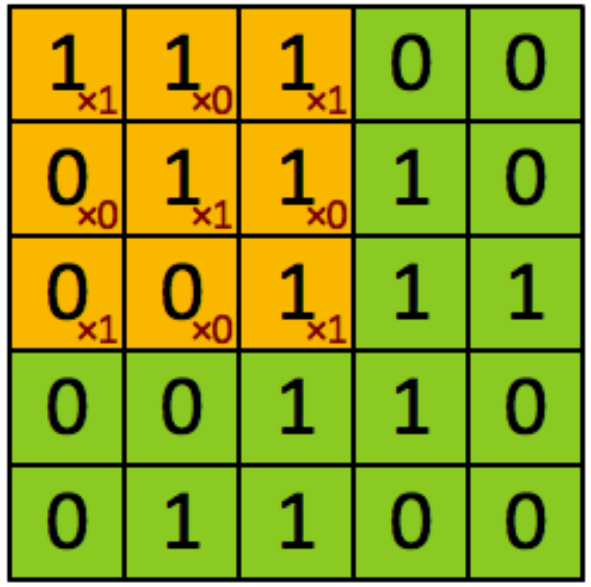


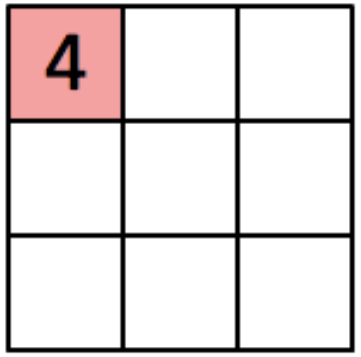
F	1	0	1
	0	1	0
	1	0	1

F

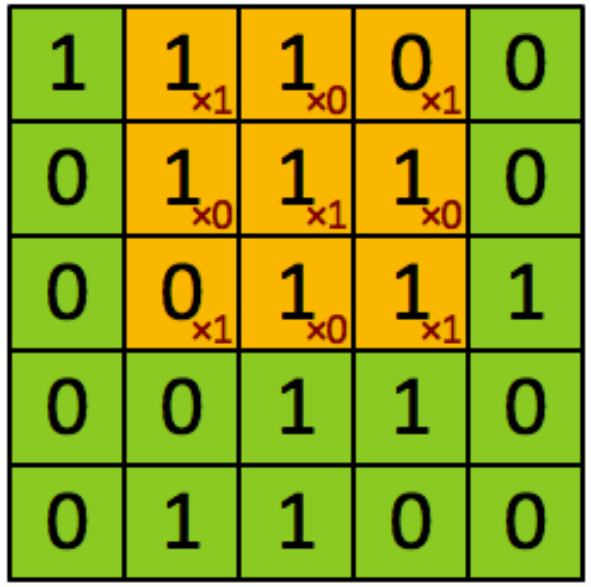
APPLY A FILTER
TO AN IMAGE

Convolutional Neural Networks

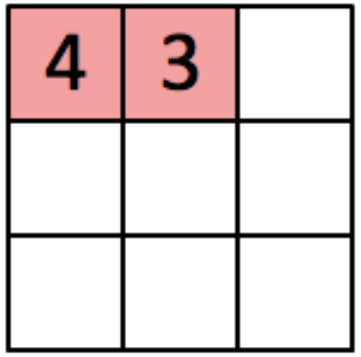




Convolutional Neural Networks

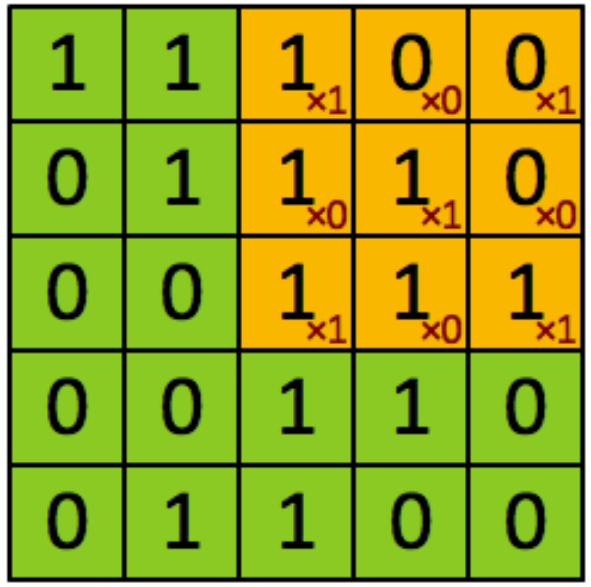


Image

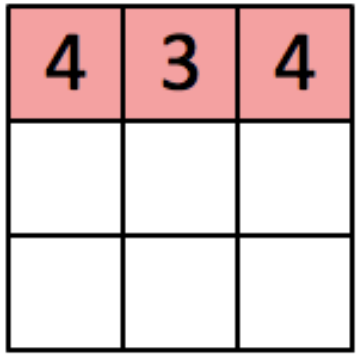


Convolved Feature

Convolutional Neural Networks

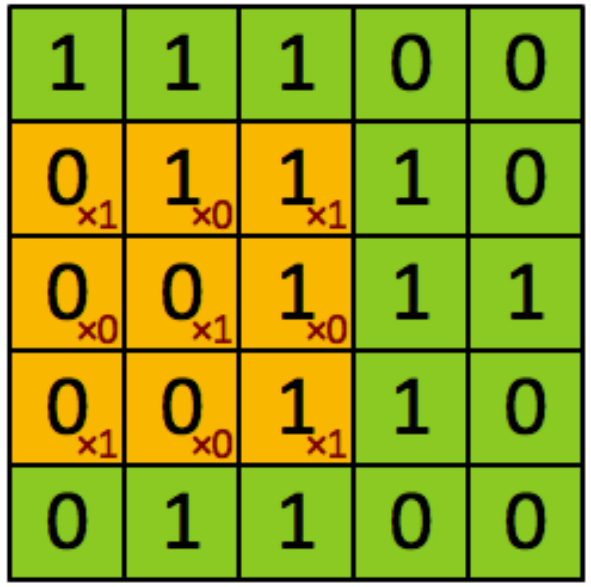


Image

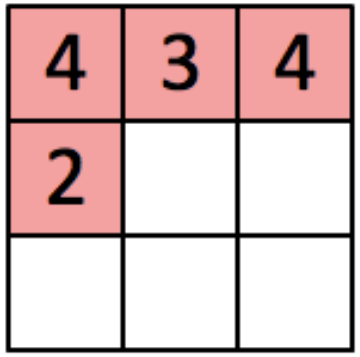


Convolved Feature

Convolutional Neural Networks

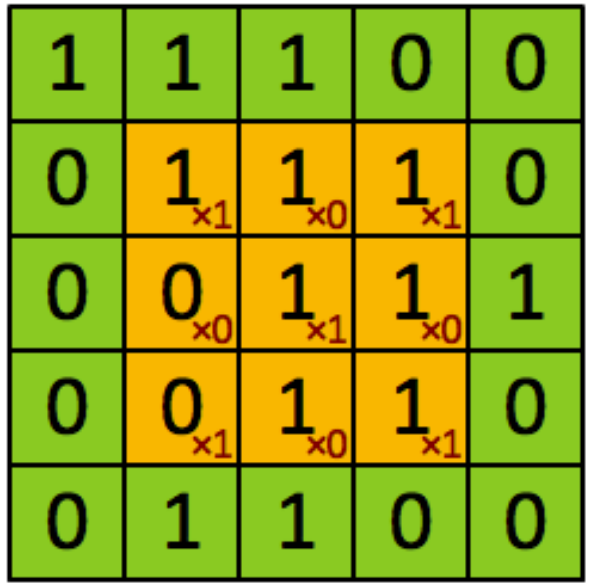


Image

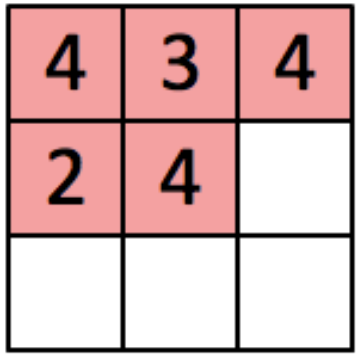


Convolved Feature

Convolutional Neural Networks

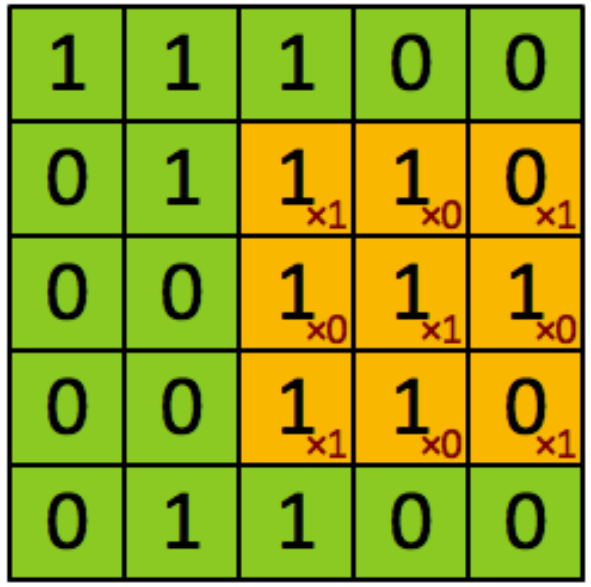


Image

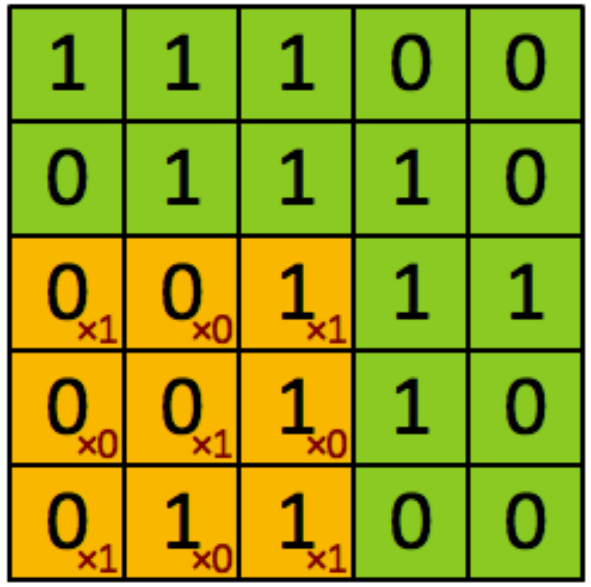


Convolved Feature

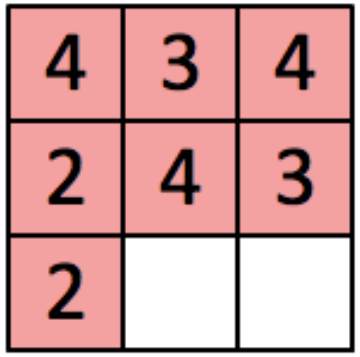
Convolutional Neural Networks



Convolutional Neural Networks



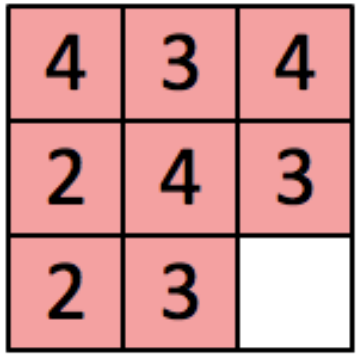
Image



Convolved Feature

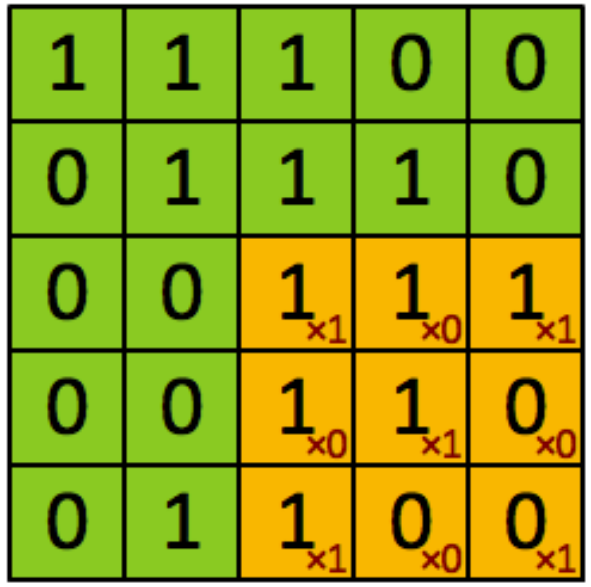
Convolutional Neural Networks

Image

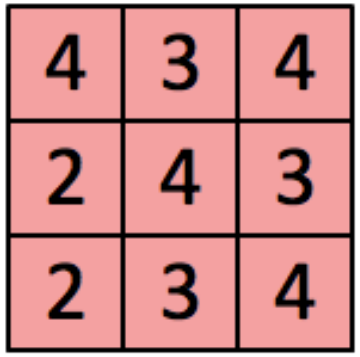


Convolved Feature

Convolutional Neural Networks

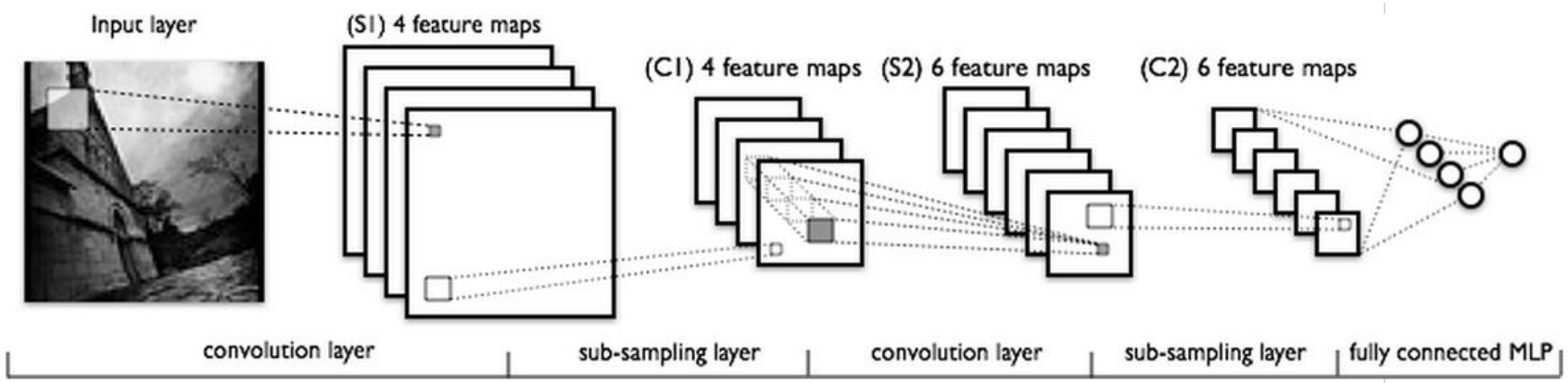


Image



Convolved Feature

Convolutional Neural Networks



CNNs are **stacks** of (convolution) layers

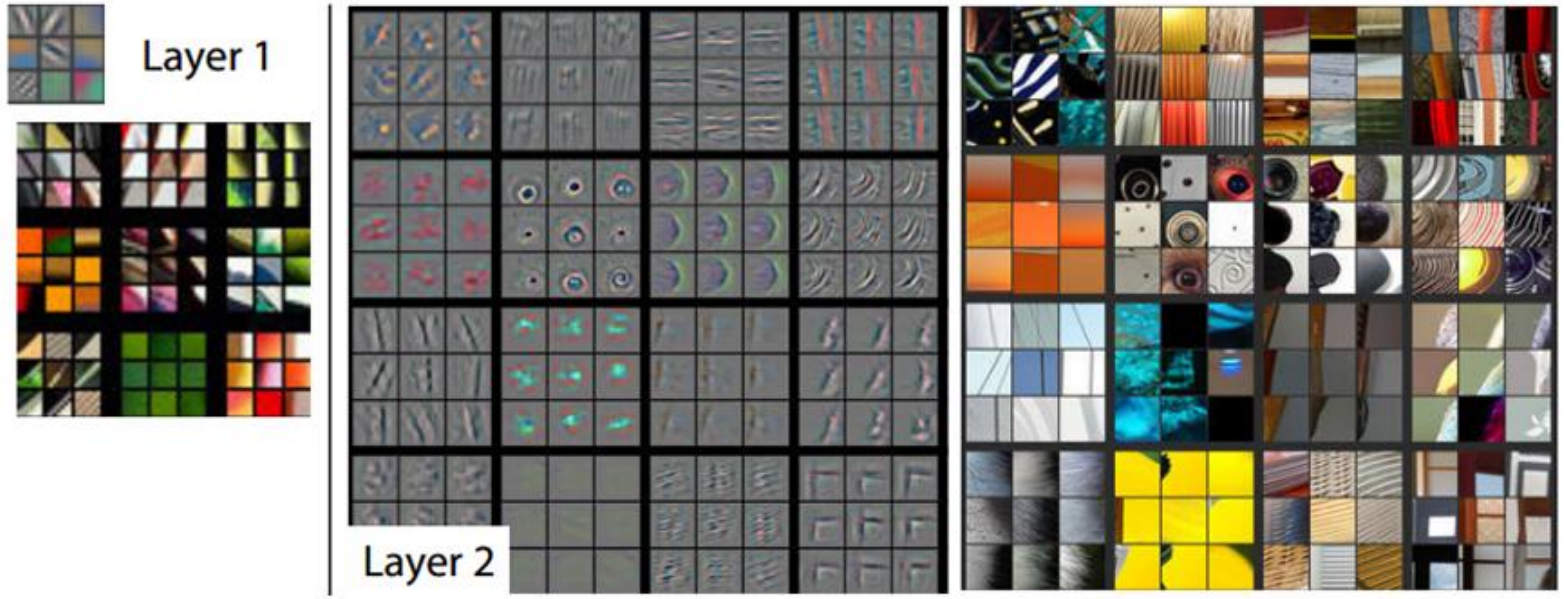
Automatic **hierarchical** feature extraction

Learning a **representation** (features) of the input

Filters are **learned** from data

Convolutional Neural Networks

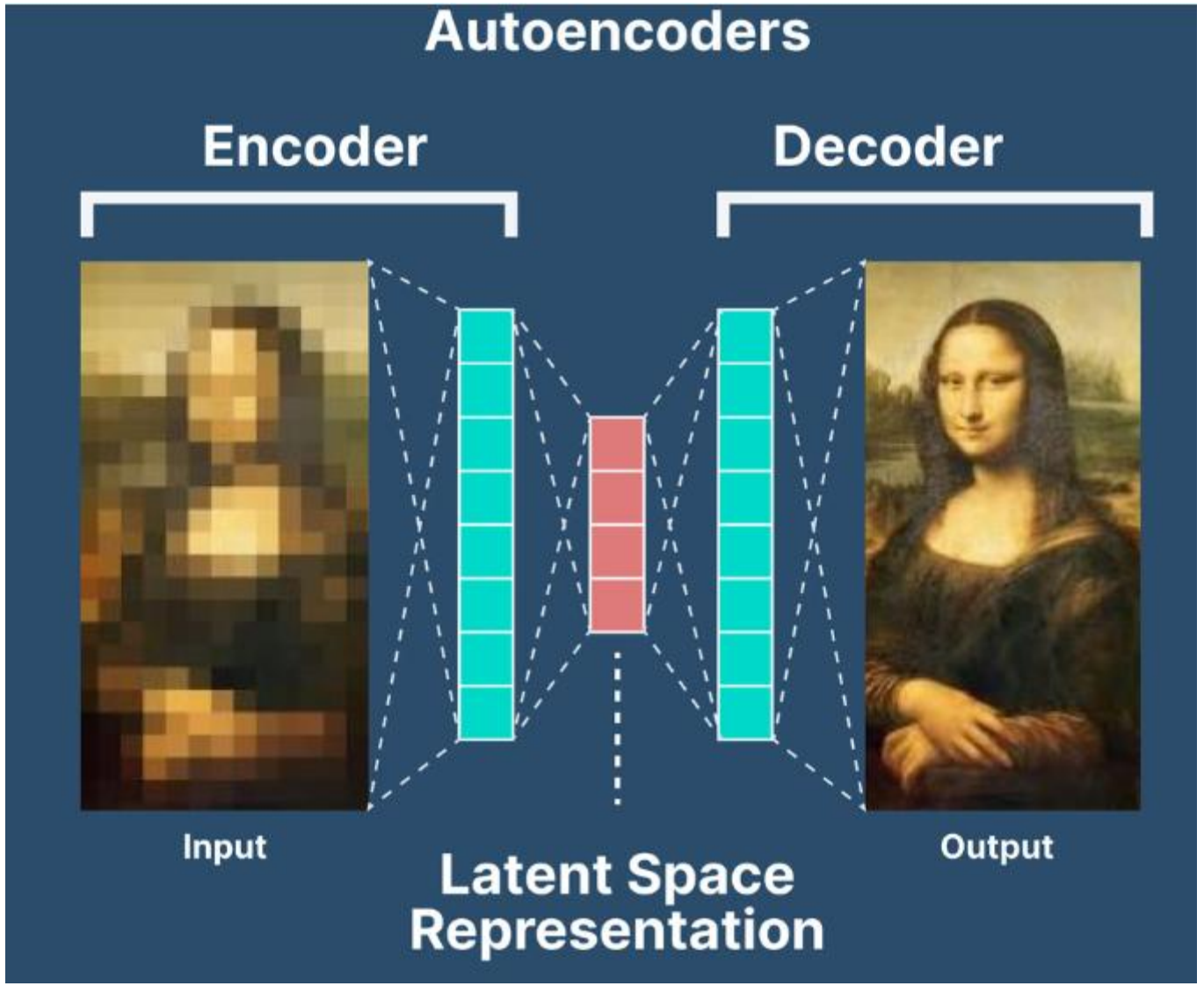
Examples of learned filters



Visualizations of Layer 1 and 2. Each layer illustrates 2 pictures, one which shows the filters themselves and one that shows what part of the image are most strongly activated by the given filter. For example, in the space labled Layer 2, we have representations of the 16 different filters (on the left)

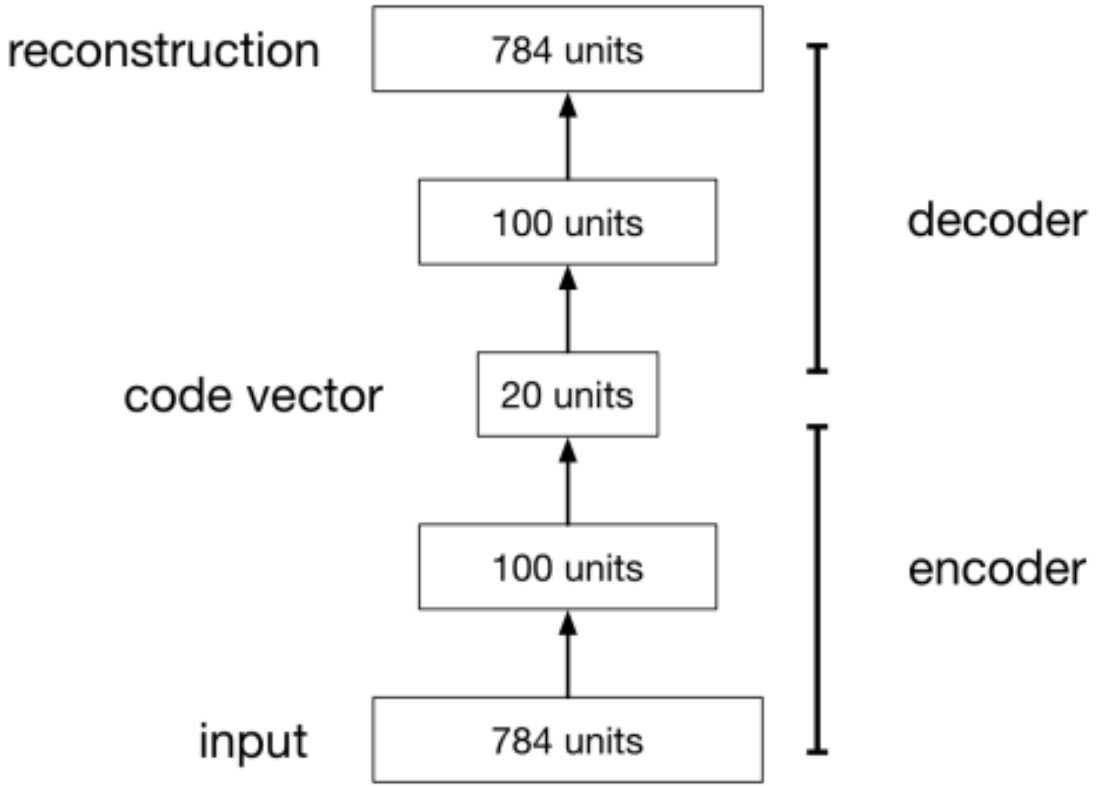
www.kdnuggets.com

Autoencoders



Autoencoders

- An autoencoder is a feed-forward neural net whose job it is to take an input **x** and predict **x**.
- To make this non-trivial, we need to add a bottleneck layer whose dimension is much smaller than the input.



Autoencoders

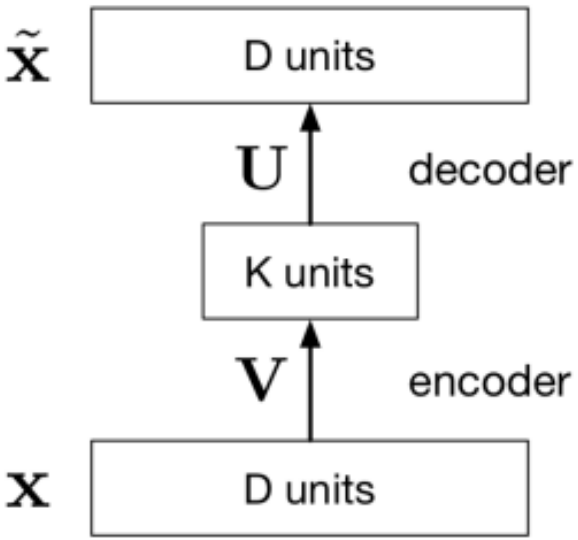
Why autoencoders?

- Map high-dimensional data to two dimensions for visualization
- Compression (i.e. reducing the file size)
- Learn abstract features in an unsupervised way so you can apply them to a supervised task
    - Unlabled data can be much more plentiful than labeled data

Principal Component Analysis

- The simplest kind of autoencoder has one hidden layer, linear activations, and squared error loss.

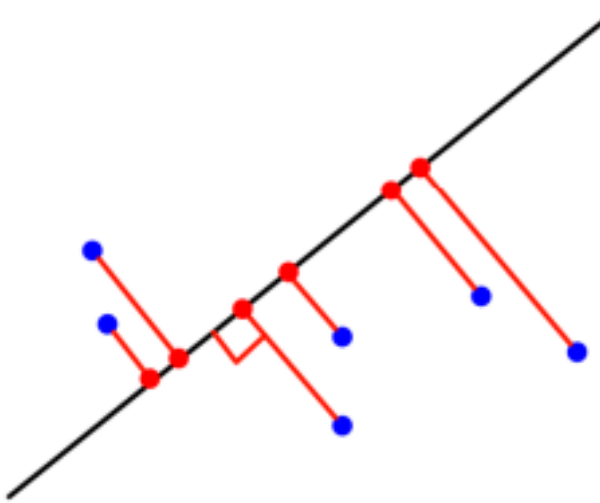
$$\mathcal{L}(\mathbf{x}, \tilde{\mathbf{x}}) = \|\mathbf{x} - \tilde{\mathbf{x}}\|^2$$



- This network computes $\tilde{\mathbf{x}} = \mathbf{UVx}$, which is a linear function.
- If $K \geq D$, we can choose $\mathbf{U}$ and $\mathbf{V}$ such that $\mathbf{UV}$ is the identity. This isn't very interesting.
- But suppose $K < D$:
  - $\mathbf{V}$ maps $\mathbf{x}$ to a K-dimensional space, so it's doing dimensionality reduction.
  - The output must lie in a K-dimensional subspace, namely the column space of $\mathbf{U}$.

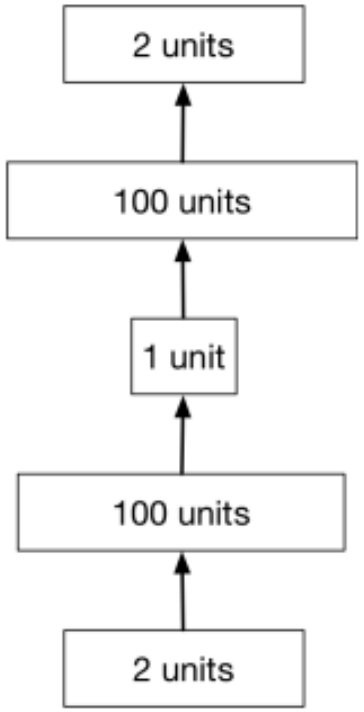
Principal Component Analysis

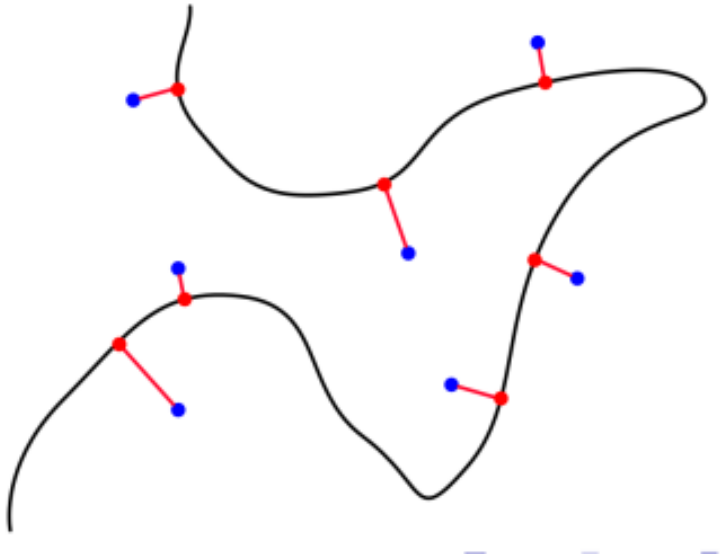
- We just saw that a linear autoencoder has to map D-dimensional inputs to a K-dimensional subspace $\mathcal{S}$.
- Knowing this, what is the best possible mapping it can choose?
  - By definition, the projection of $\mathbf{x}$ onto $\mathcal{S}$ is the point in $\mathcal{S}$ which minimizes the distance to $\mathbf{x}$.



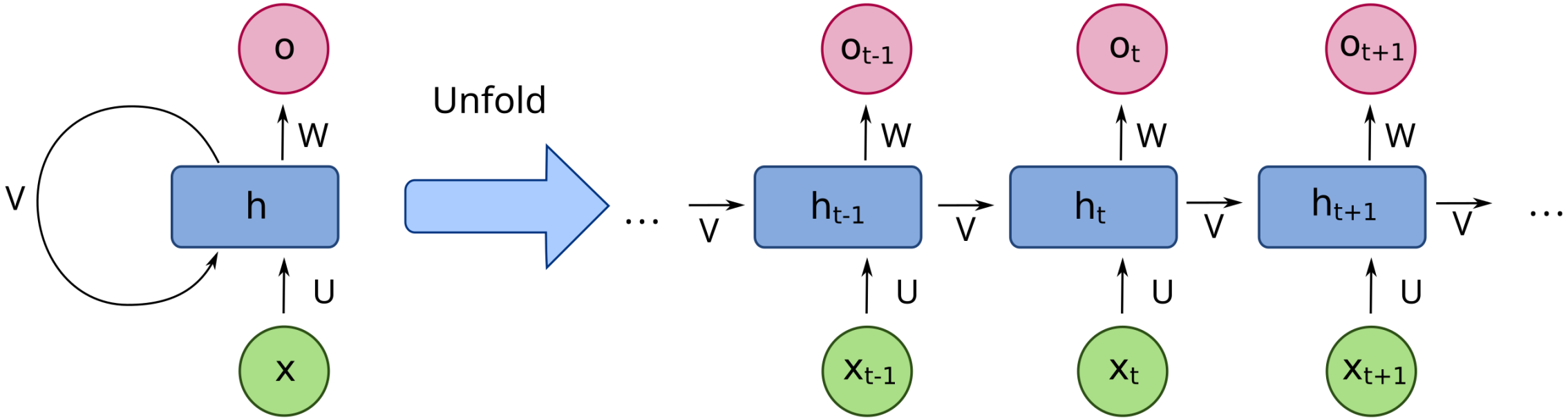
Deep Autoencoders

- Deep nonlinear autoencoders learn to project the data, not c subspace, but onto a nonlinear manifold
- This manifold is the image of the decoder.
- This is a kind of nonlinear dimensionality reduction.



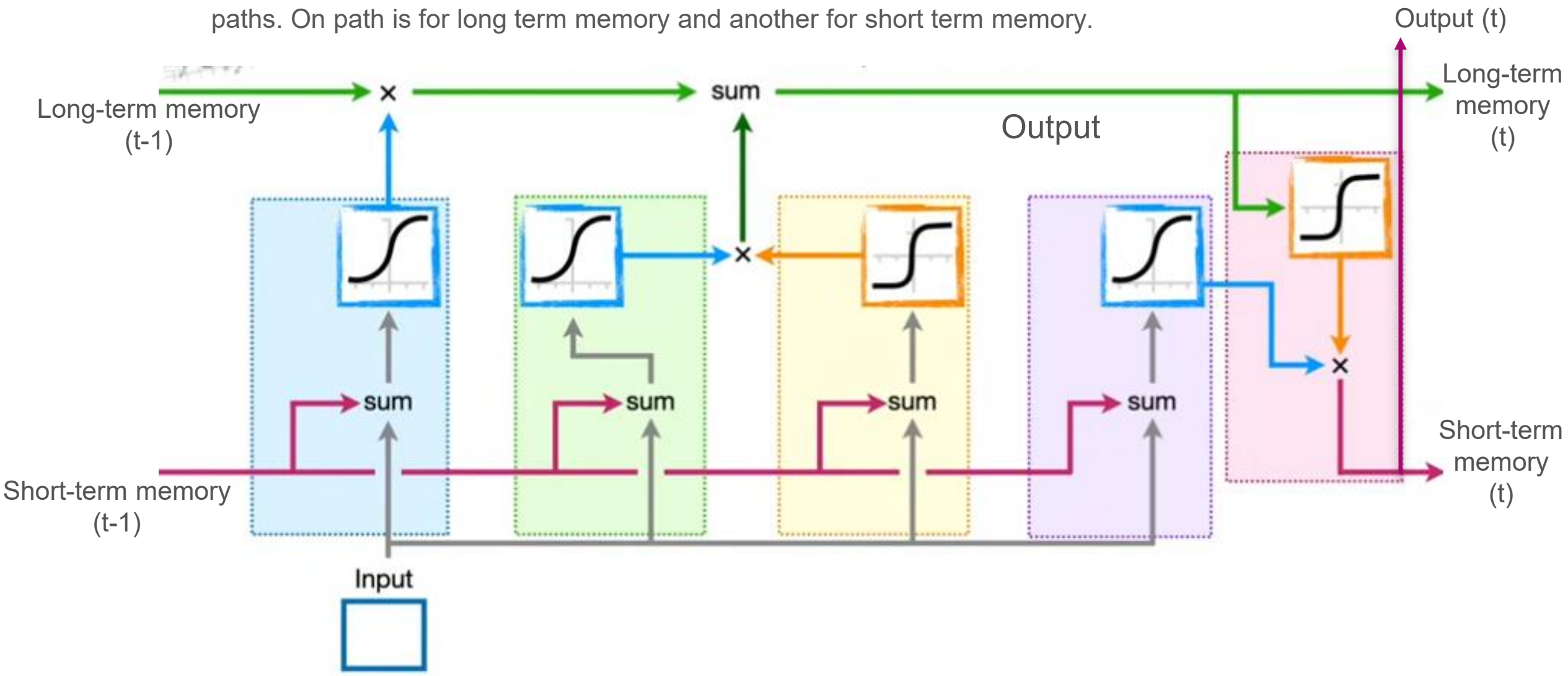


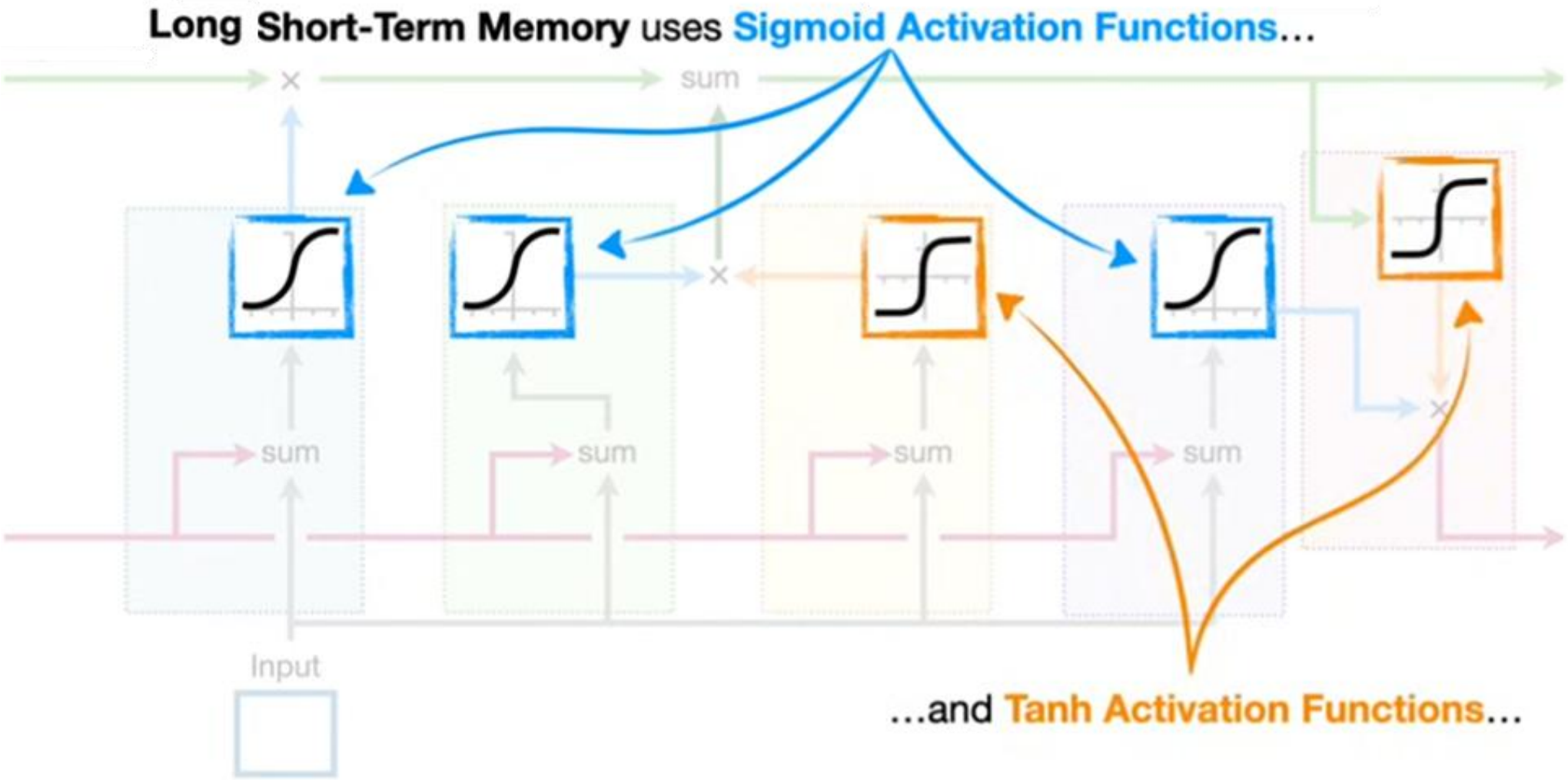
RNN – See Notebook

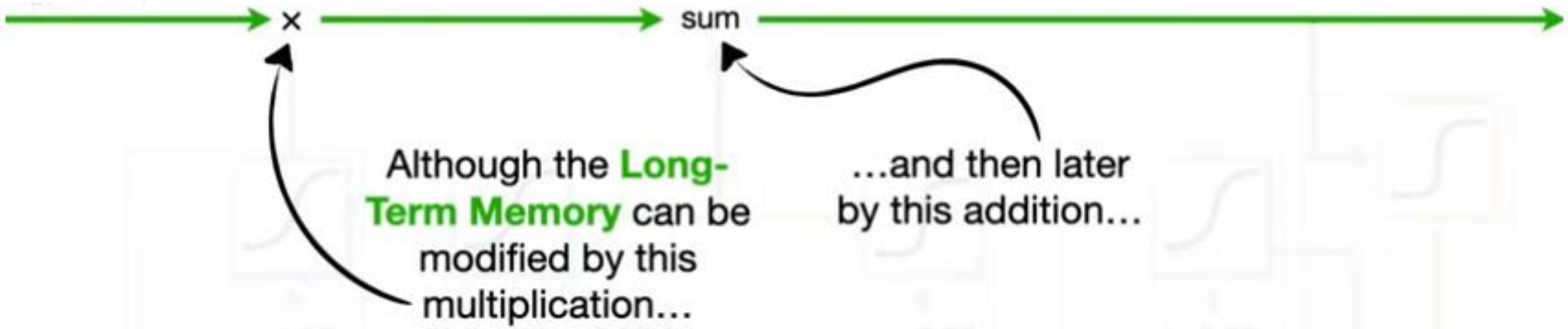


LSTM – Long Short Term Memory

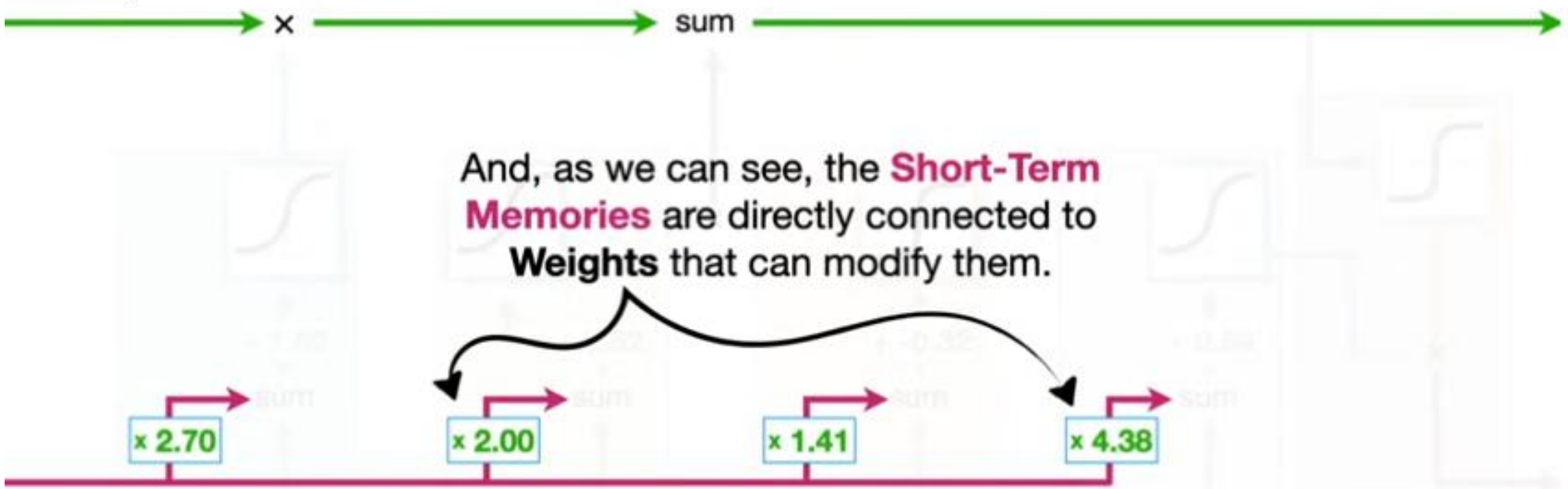
- **Main idea:** instead of using the same feedback loop connection for events (data) that happened long ago and events (data) that happened yesterday (e.g., previous time stamp), LSTM uses two distinct paths. On path is for long term memory and another for short term memory.

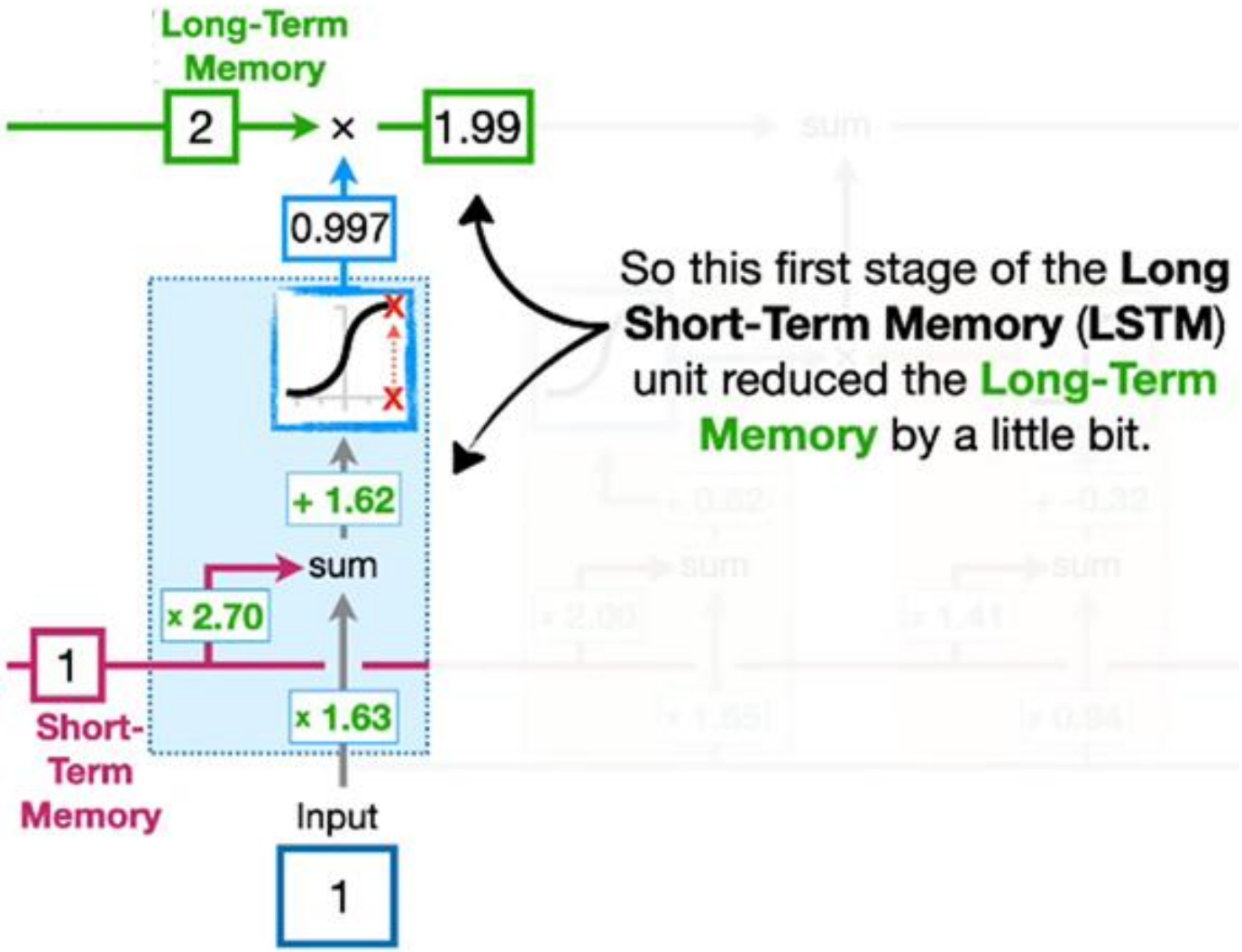


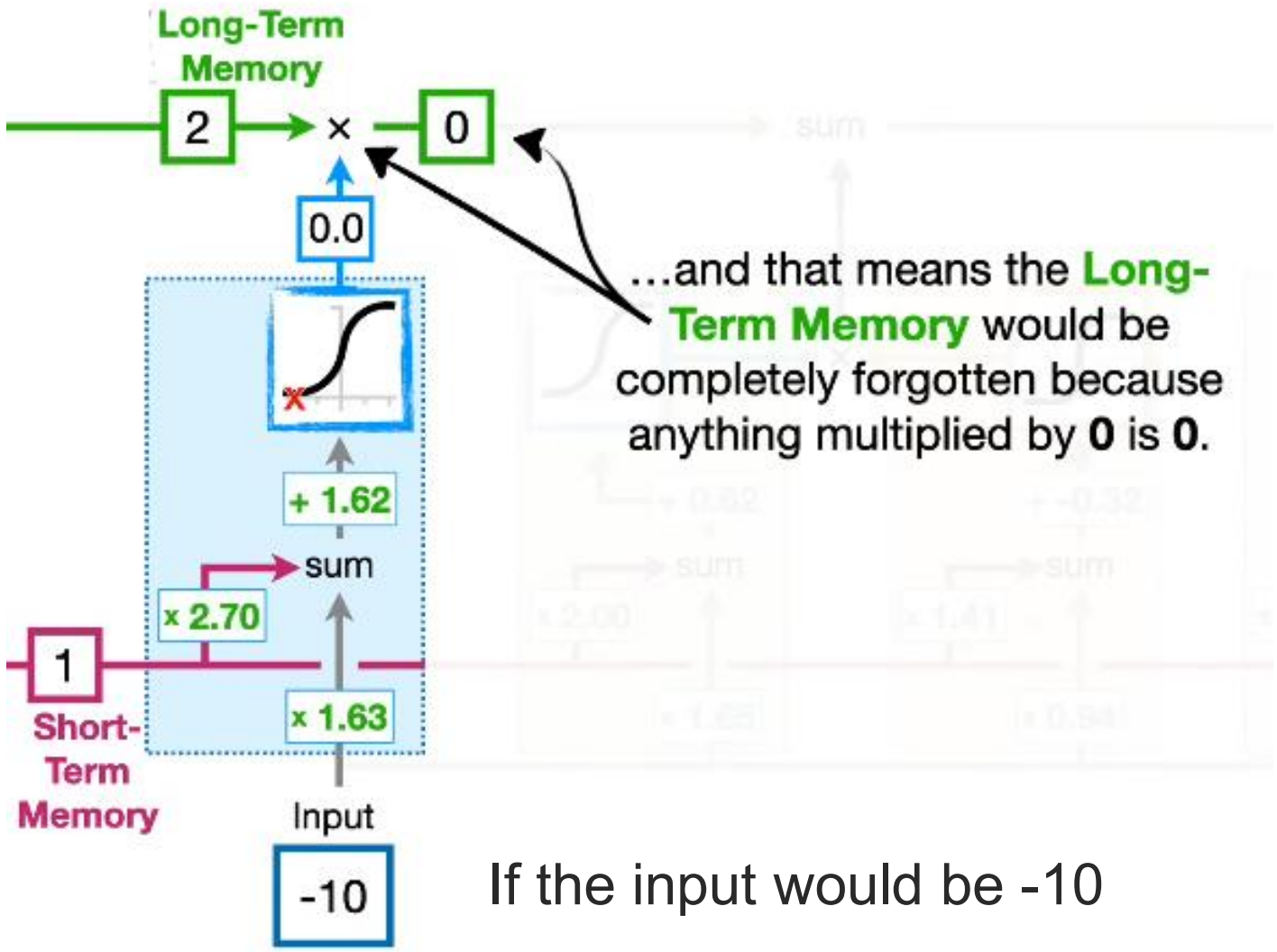

Long Short-Term Memory uses Sigmoid Activation Functions…
sum
sum
sum
sum
Input
…and Tanh Activation Functions…



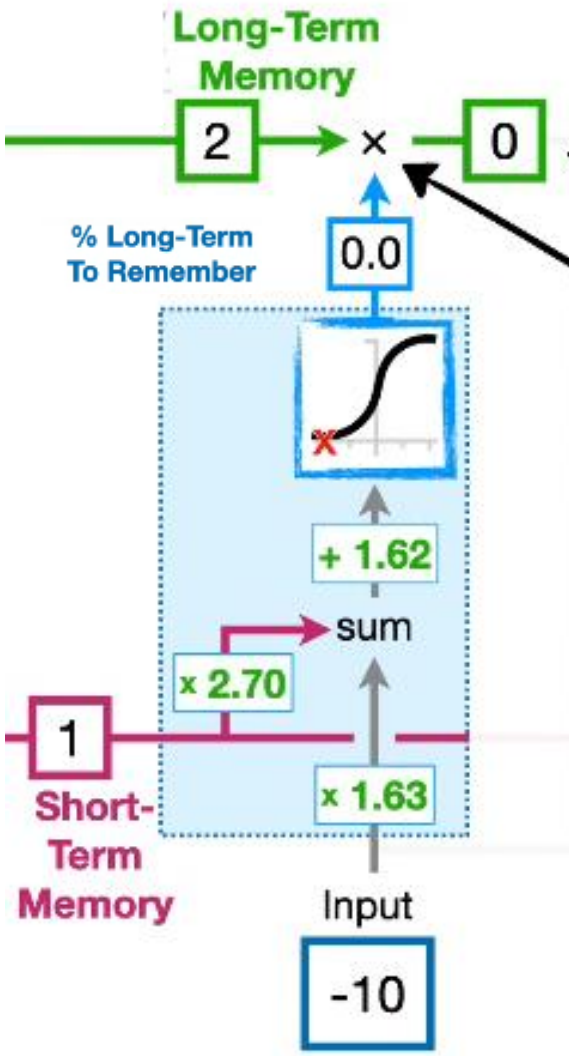
The lack of **Weights** allows the **Long-Term Memories** to flow through a series of unrolled units without causing the gradient to **explode** or **vanish**.


×
sum
And, as we can see, the Short-Term Memories are directly connected to Weights that can modify them.
x 2.70
x 2.00
x 1.41
x 4.38


Long-Term Memory
2
×
1.99
0.997
So this first stage of the Long Short-Term Memory (LSTM) unit reduced the Long-Term Memory by a little bit.
+ 1.62
sum
× 2.70
1
Short-Term Memory
× 1.63
Input
1



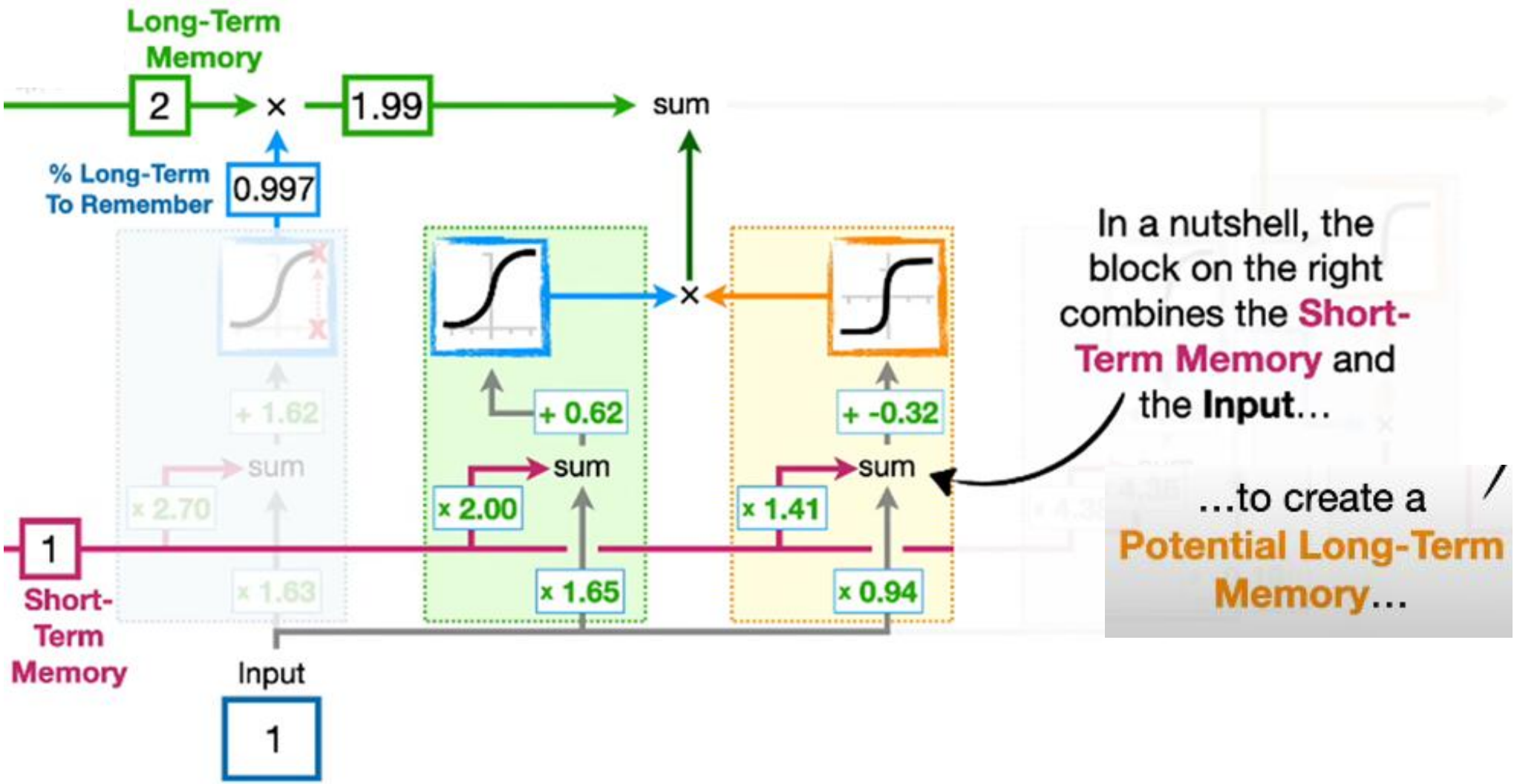
If the input would be -10

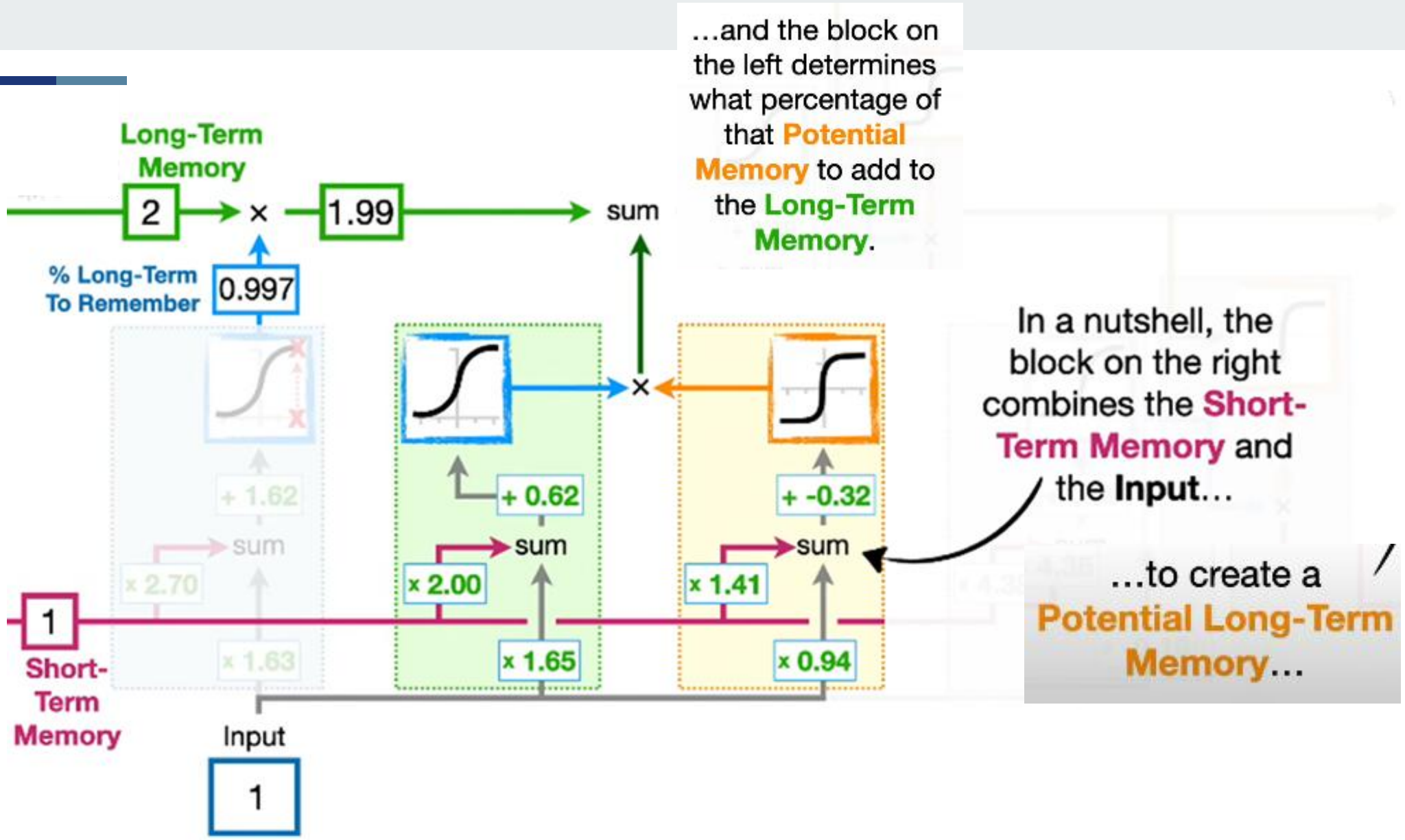


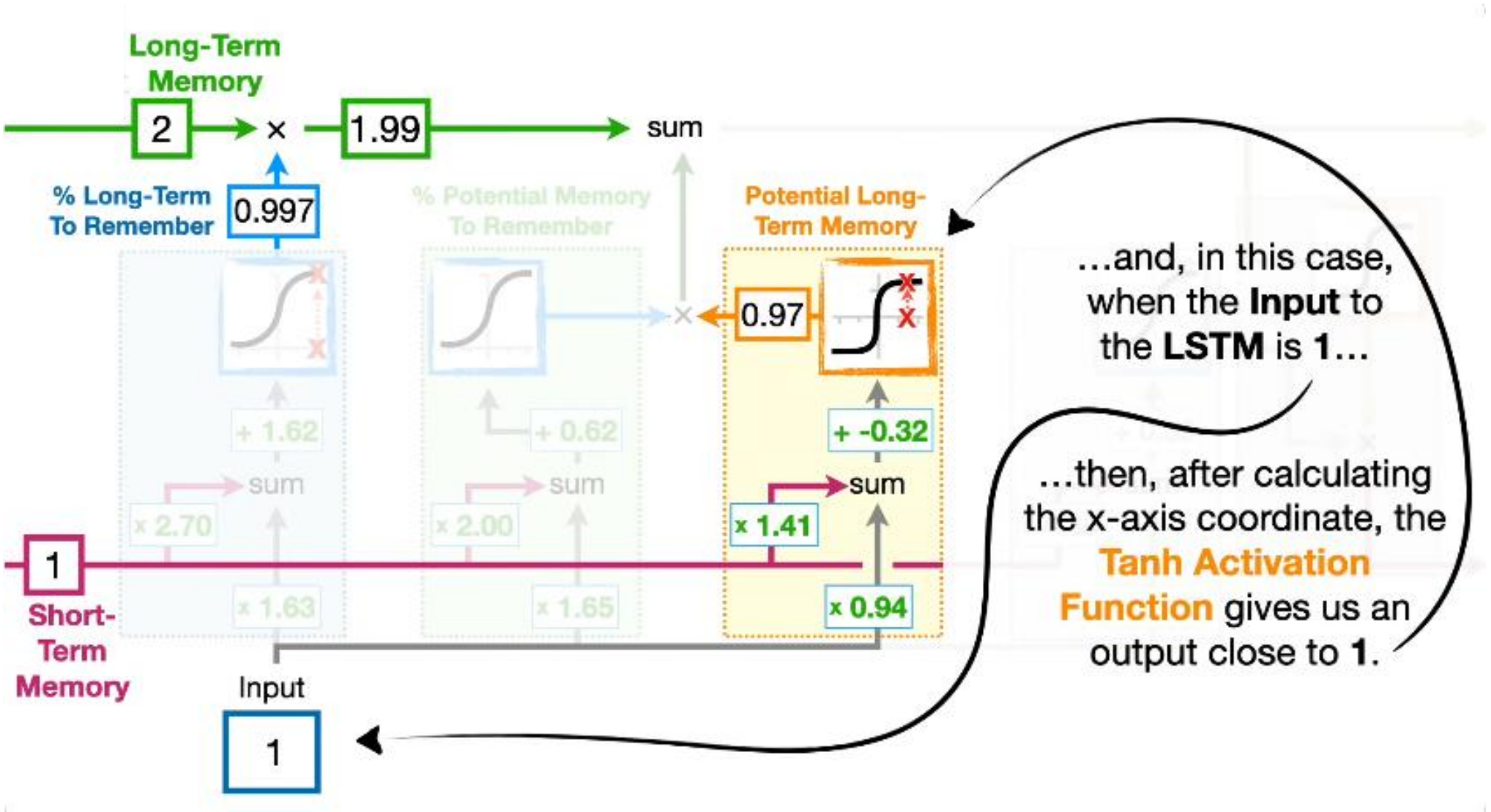
…and that means the **Long-Term Memory** would be completely forgotten because anything multiplied by **0** is **0**.

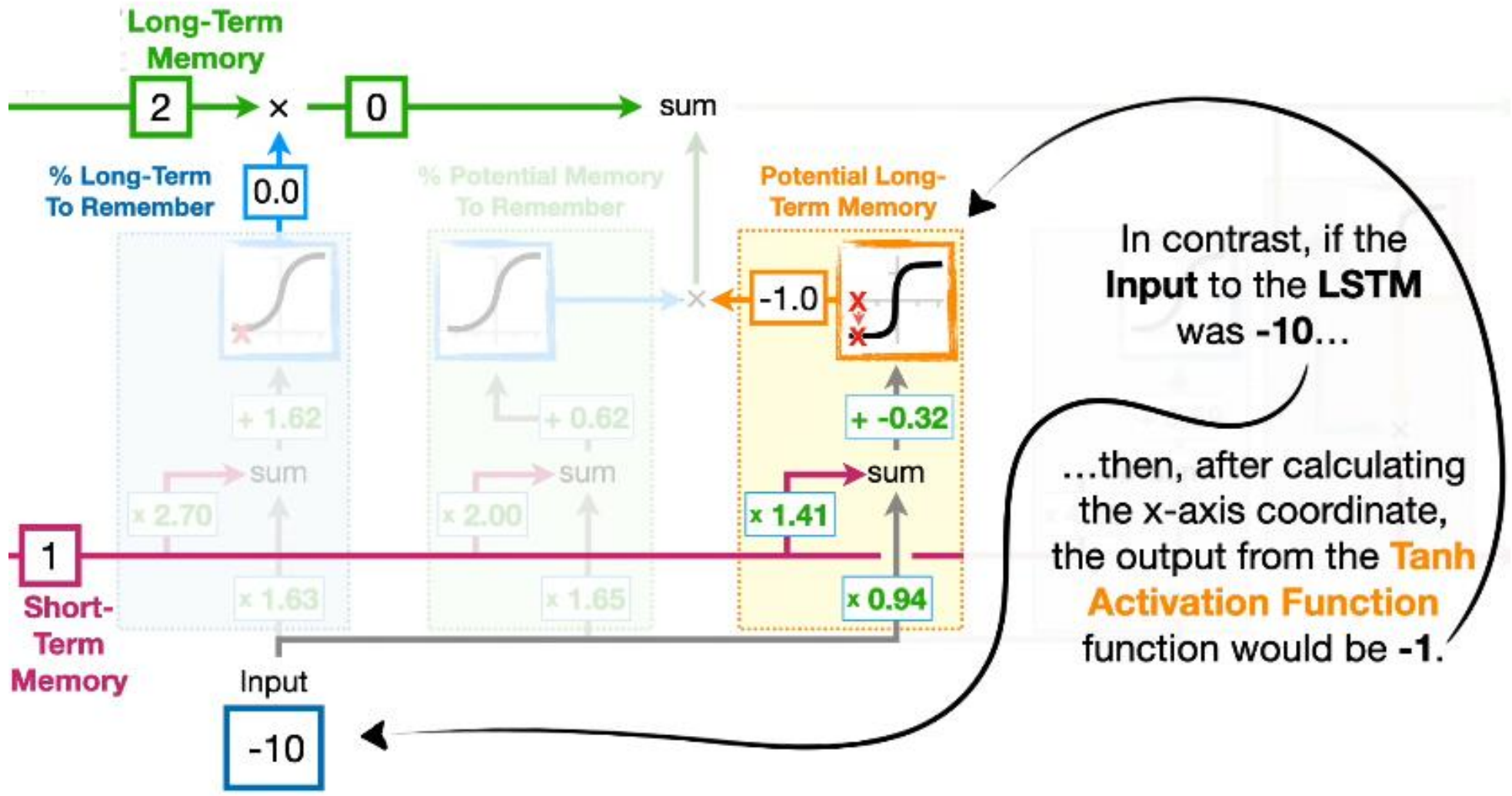
To summarize, the first stage in a **Long Short-Term Memory** unit determines what percentage of the **Long-Term Memory** is remembered.

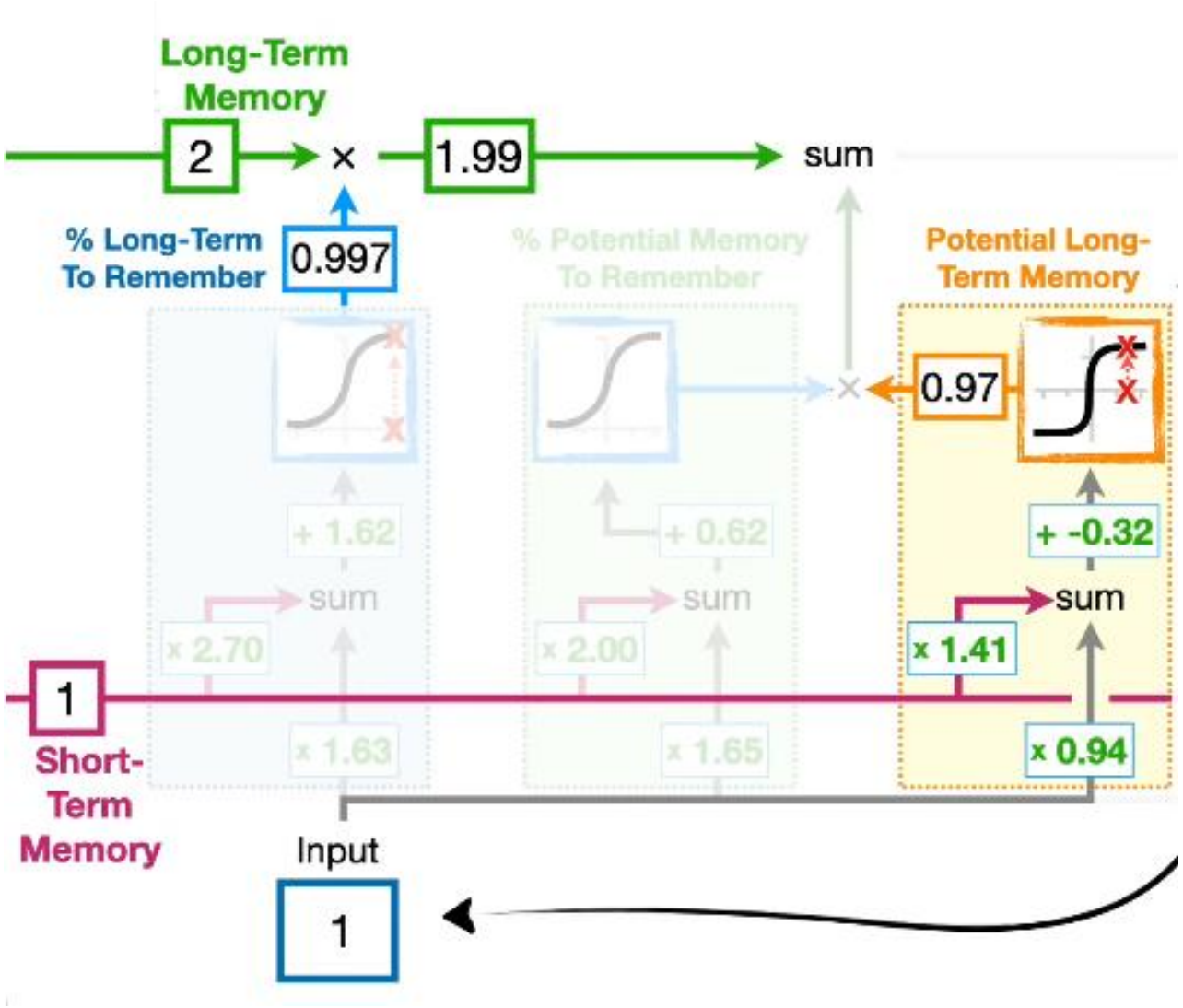
Therefore, this part is called the **Forget gate**


Long-Term Memory
2
x
1.99
sum
% Long-Term To Remember
0.997
+ 1.62
sum
x 2.70
x 1.63
+ 0.62
sum
x 2.00
x 1.65
x
+ -0.32
sum
x 1.41
x 0.94
1
Short-Term Memory
Input
1
In a nutshell, the block on the right combines the Short-Term Memory and the Input…
…to create a Potential Long-Term Memory…


Long-Term Memory
2
×
1.99
sum
% Long-Term To Remember
0.997
…and the block on the left determines what percentage of that Potential Memory to add to the Long-Term Memory.
In a nutshell, the block on the right combines the Short-Term Memory and the Input…
+ 1.62
sum
× 2.70
× 1.63
+ 0.62
sum
× 2.00
× 1.65
×
+ -0.32
sum
× 1.41
× 0.94
1
Short-Term Memory
Input
1
…to create a Potential Long-Term Memory…

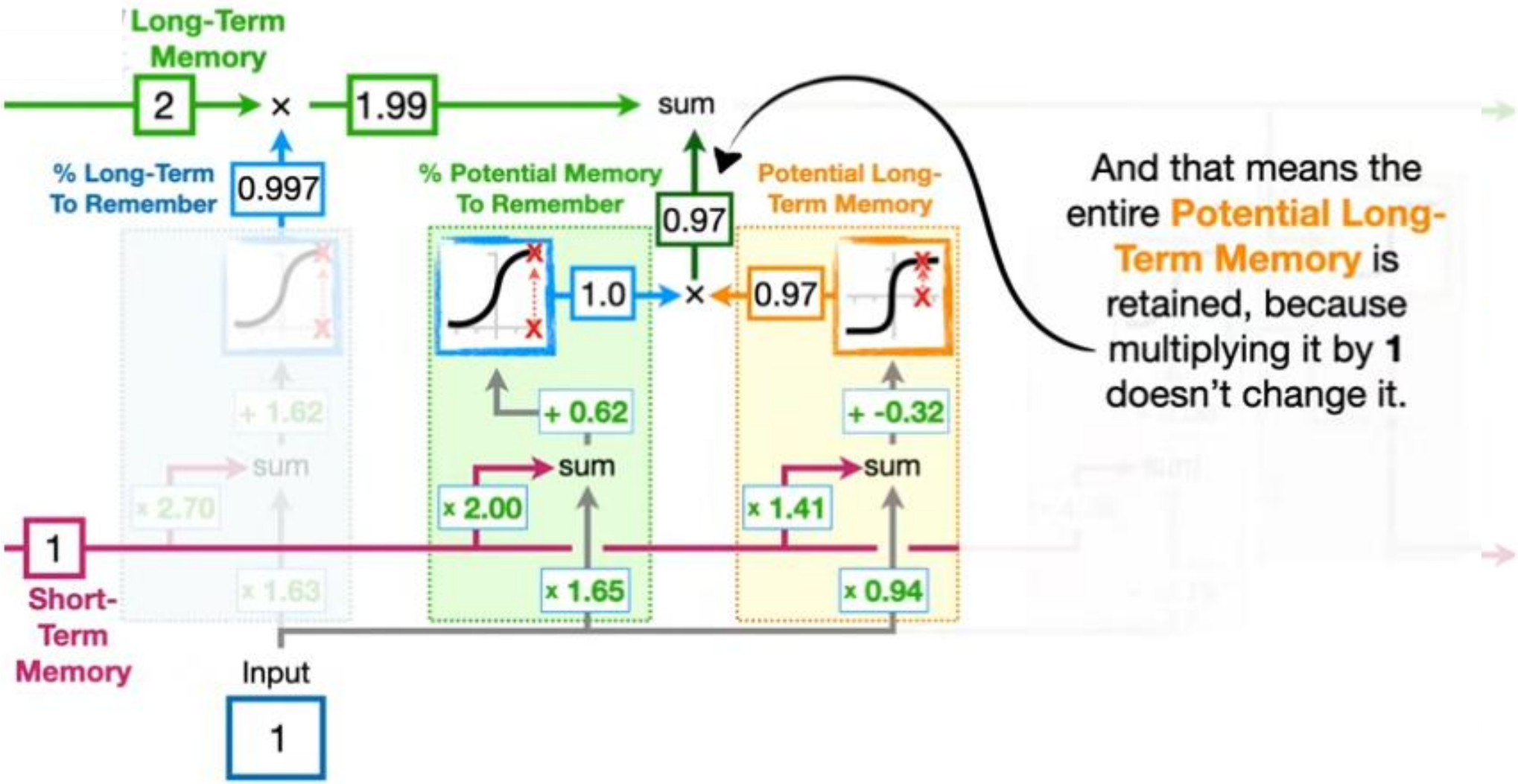

Long-Term Memory
2
×
1.99
sum
% Long-Term To Remember
0.997
% Potential Memory To Remember
Potential Long-Term Memory
0.97
+ 1.62
+ 0.62
+ -0.32
sum
sum
sum
x 2.70
x 2.00
x 1.41
1
Short-Term Memory
x 1.63
x 1.65
x 0.94
Input
1
…and, in this case, when the Input to the LSTM is 1…
…then, after calculating the x-axis coordinate, the Tanh Activation Function gives us an output close to 1.

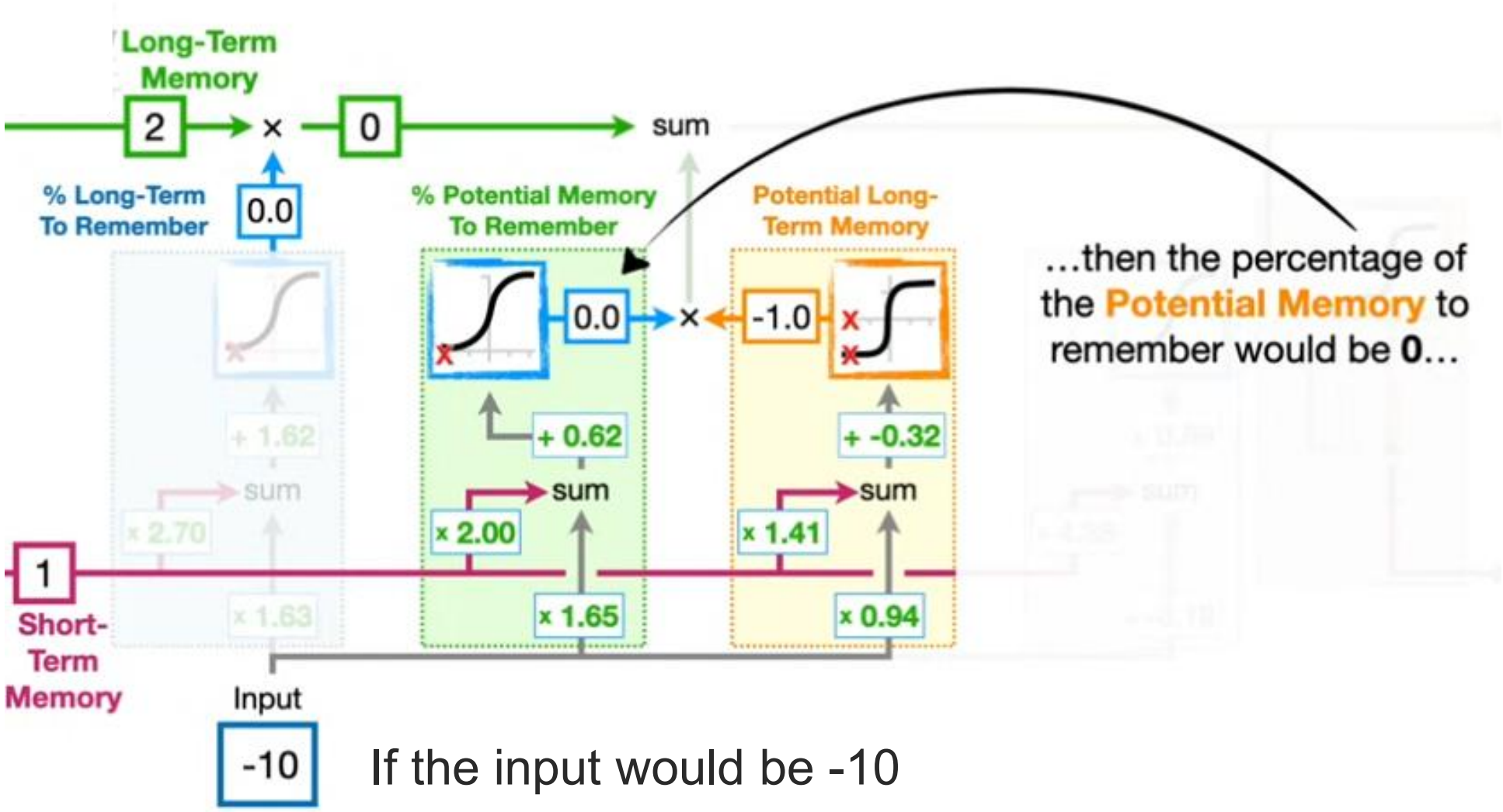

Long-Term Memory
2
×
0
sum
% Long-Term To Remember
0.0
% Potential Memory To Remember
Potential Long-Term Memory
-1.0
+ 1.62
+ 0.62
+ -0.32
sum
sum
sum
x 2.70
x 2.00
x 1.41
1
x 1.63
x 1.65
x 0.94
Short-Term Memory
Input
-10
In contrast, if the Input to the LSTM was -10...
...then, after calculating the x-axis coordinate, the output from the Tanh Activation Function function would be -1.



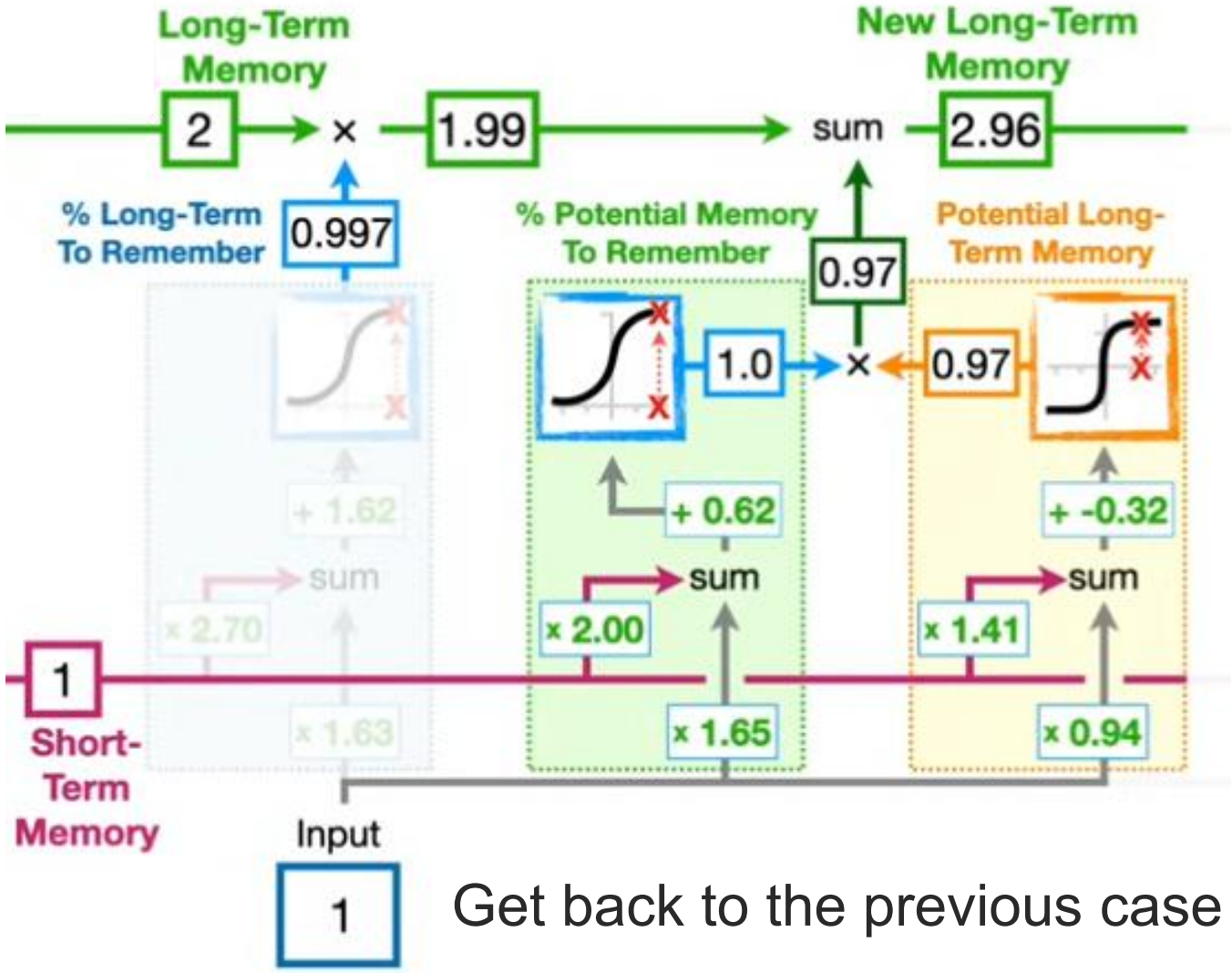
Get back to the previous case

Now the **LSTM** has to decide how much of this **Potential Memory** to save…


Long-Term Memory
2
×
1.99
sum
% Long-Term To Remember
0.997
% Potential Memory To Remember
0.97
Potential Long-Term Memory
1.0
×
0.97
+ 0.62
+ -0.32
sum
sum
x 2.00
x 1.41
1
x 1.65
x 0.94
Short-Term Memory
Input
1
And that means the entire Potential Long-Term Memory is retained, because multiplying it by 1 doesn't change it.



If the input would be -10



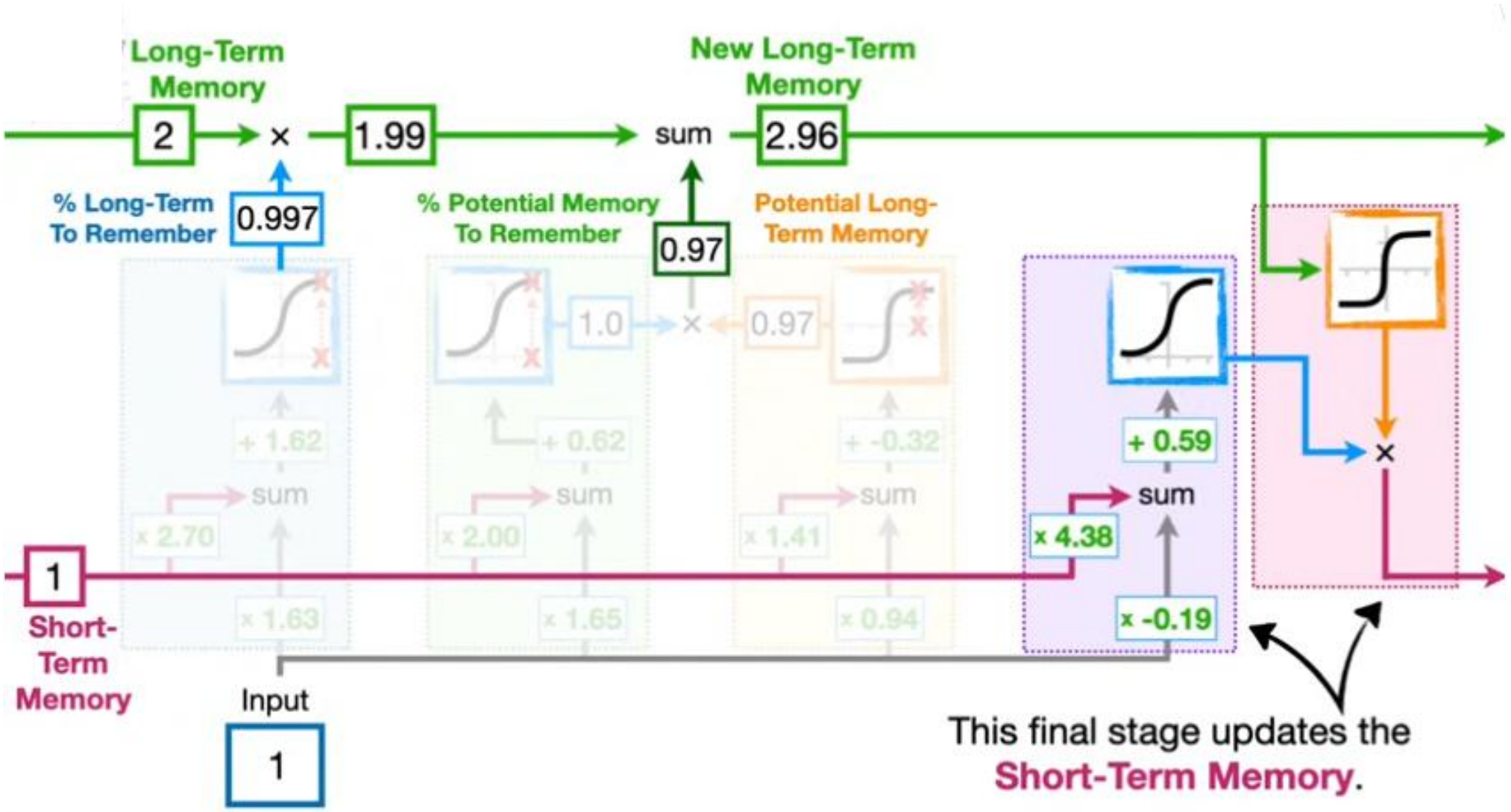
Get back to the previous case

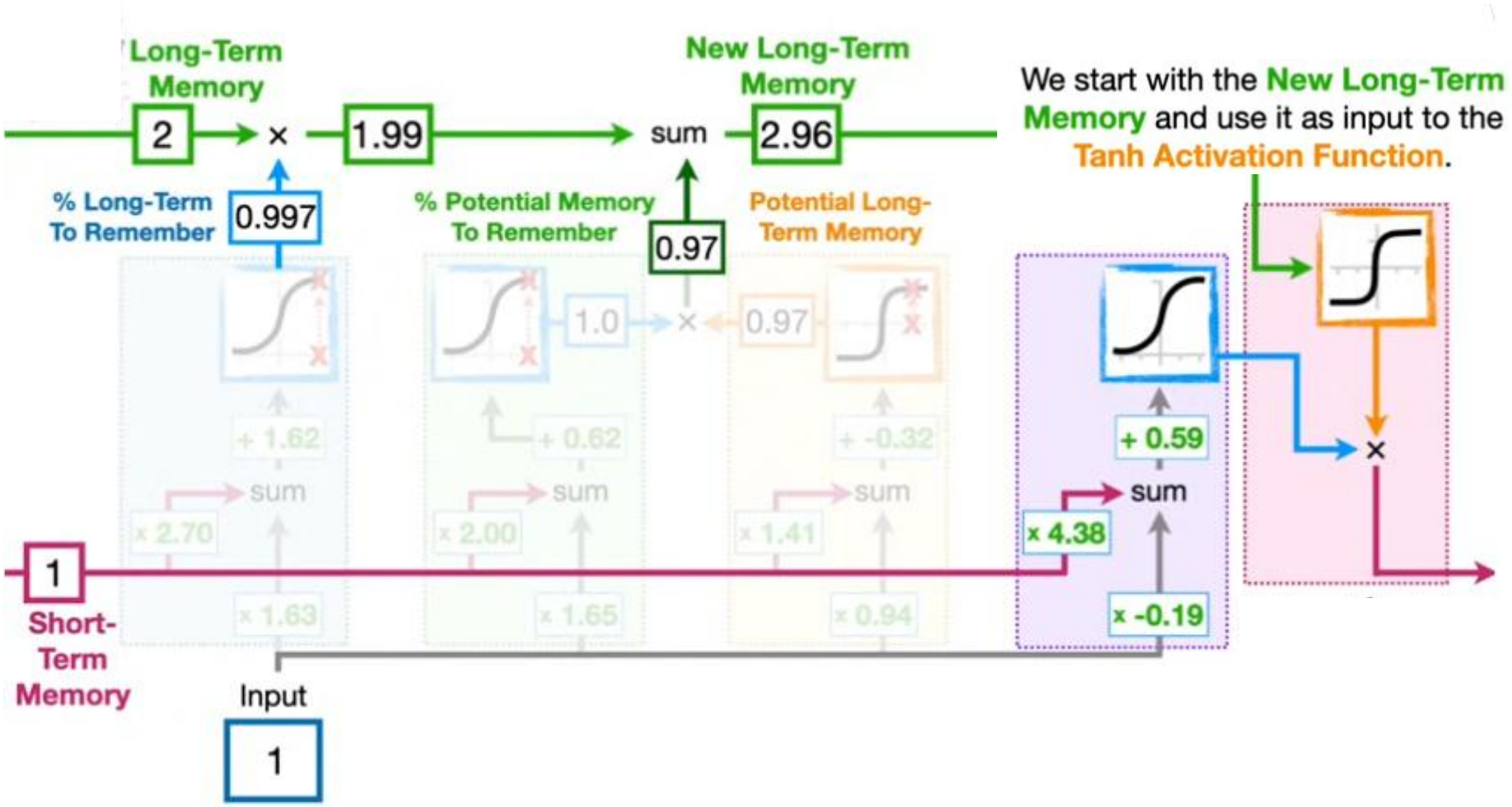
…we add **0.97** to the existing **Long-Term Memory**…

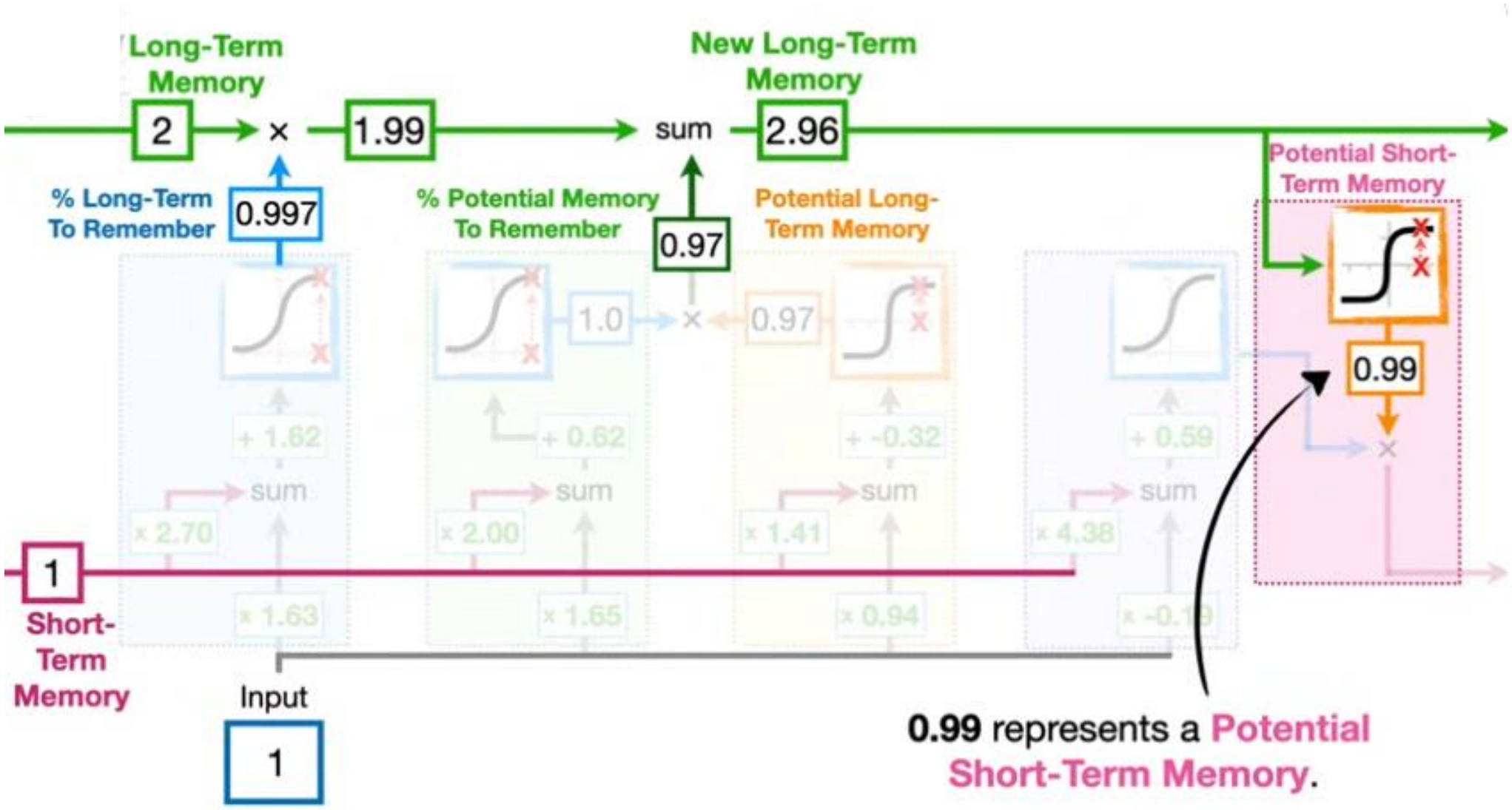
…and we get a new **Long-Term Memory**, **2.96**.

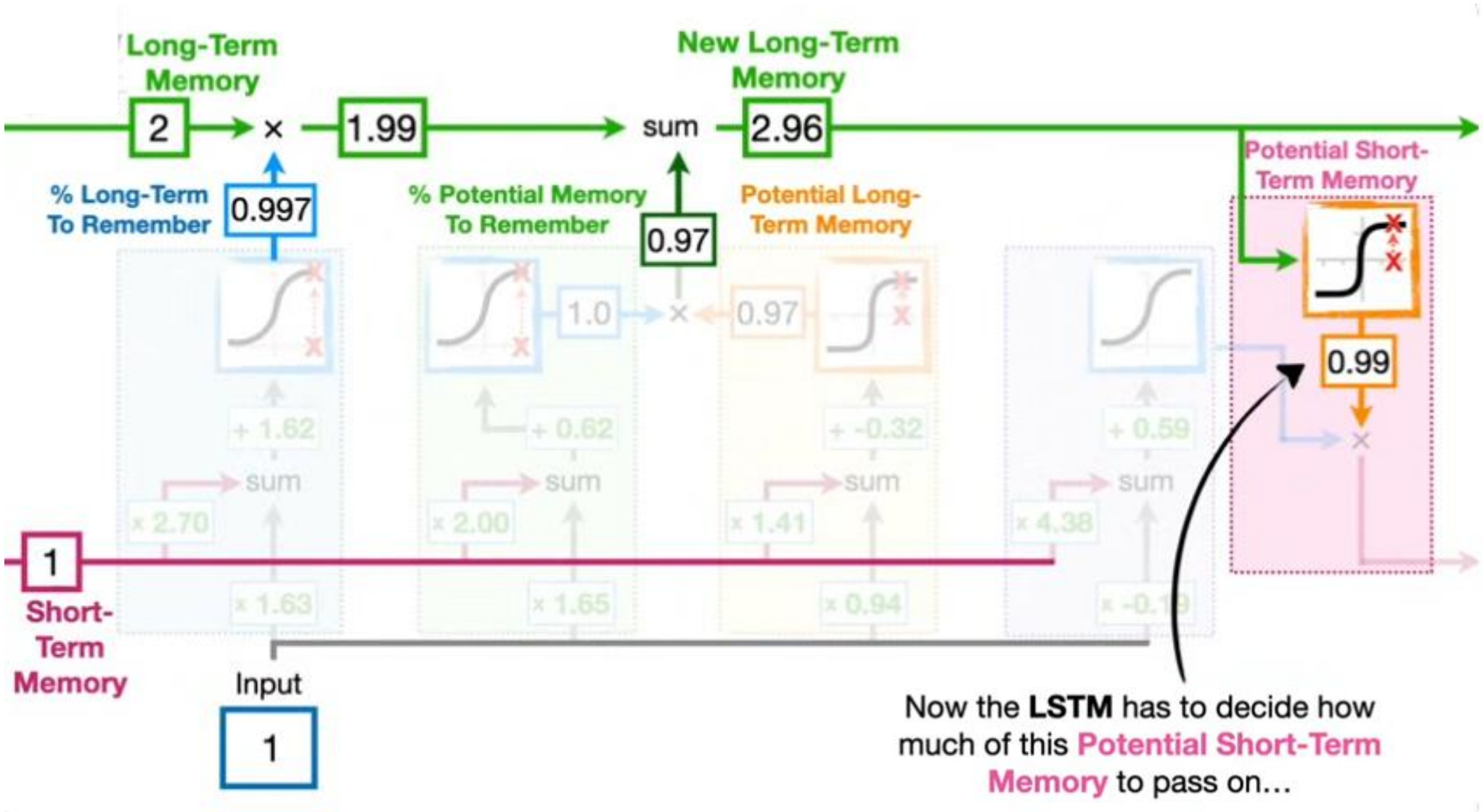
Even though this part of the **Long Short-Term Memory** unit determines how we should update the **Long-Term Memory**…

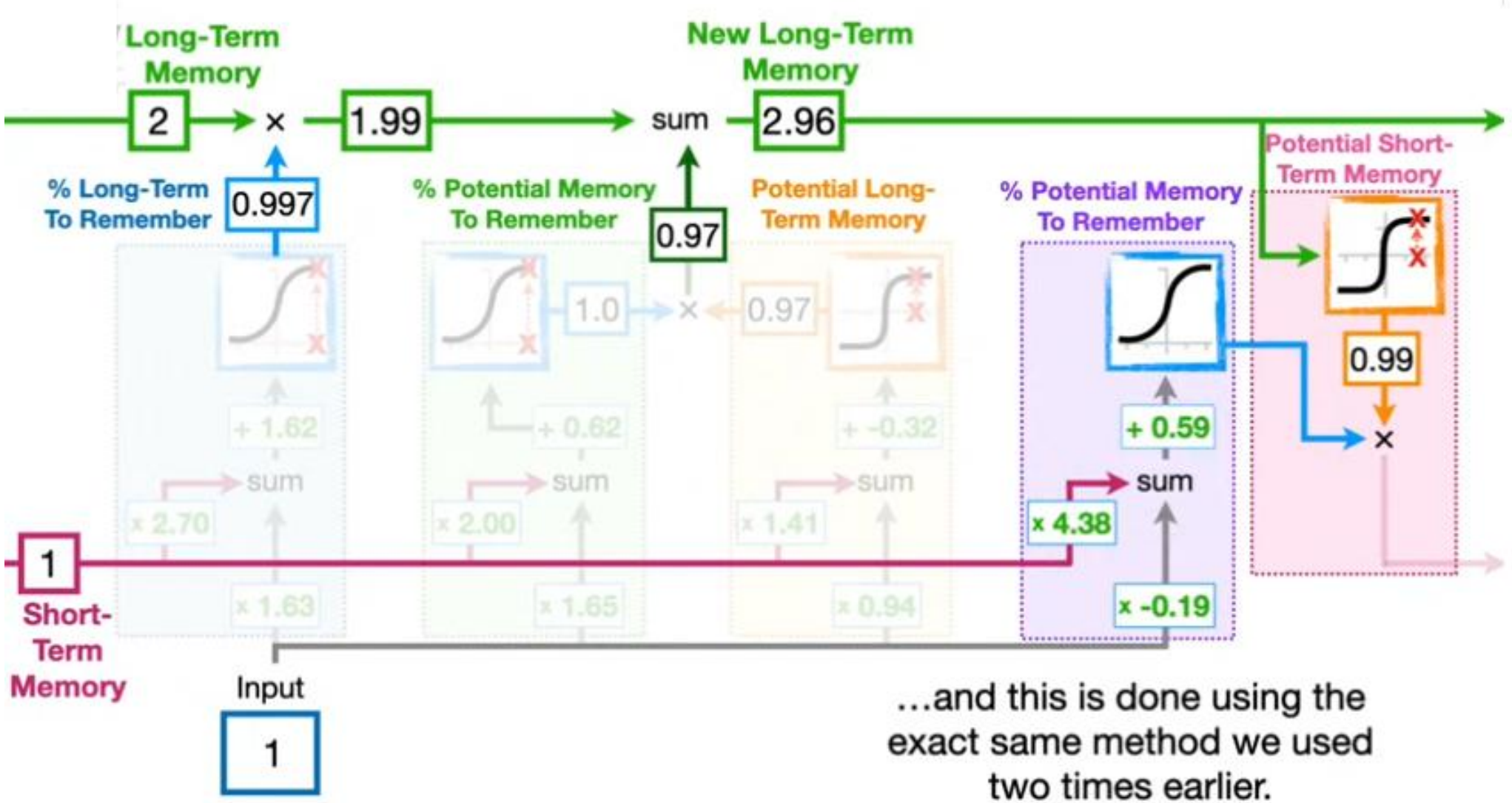
…it is usually called the **Input Gate**.

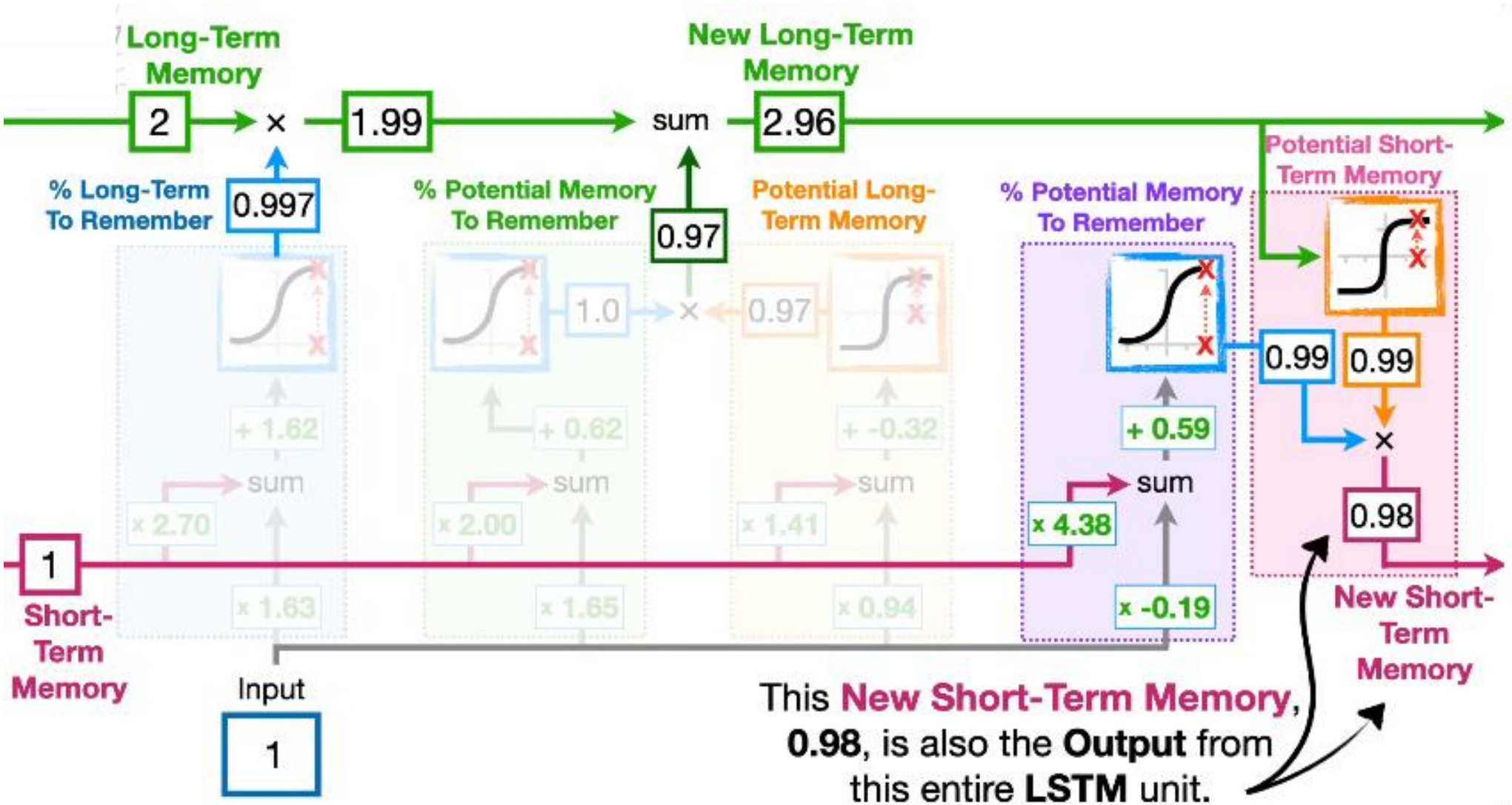

Long-Term Memory
2
×
1.99
sum
New Long-Term Memory
2.96
% Long-Term To Remember
0.997
% Potential Memory To Remember
0.97
Potential Long-Term Memory
1.0
×
0.97
+ 1.62
+ 0.62
+ -0.32
sum
sum
sum
x 2.70
x 2.00
x 1.41
x 1.63
x 1.65
x 0.94
+ 0.59
sum
x 4.38
x -0.19
×
1
Short-Term Memory
Input
1
This final stage updates the Short-Term Memory.


Long-Term Memory
2
×
1.99
New Long-Term Memory
sum
2.96
We start with the New Long-Term Memory and use it as input to the Tanh Activation Function.
% Long-Term To Remember
0.997
% Potential Memory To Remember
0.97
Potential Long-Term Memory
1.0
×
0.97
+ 1.62
+ 0.62
+ -0.32
+ 0.59
sum
sum
sum
sum
x 2.70
x 2.00
x 1.41
x 4.38
1
Short-Term Memory
x 1.63
x 1.65
x 0.94
x -0.19
Input
1


Long-Term Memory
2
×
1.99
New Long-Term Memory
sum
2.96
% Long-Term To Remember
0.997
% Potential Memory To Remember
0.97
Potential Long-Term Memory
Potential Short-Term Memory
0.99
1.0
0.97
+ 1.62
+ 0.62
+ -0.32
+ 0.59
sum
× 2.70
× 2.00
× 1.41
× 4.38
× 1.63
× 1.65
× 0.94
1
Short-Term Memory
Input
1
0.99 represents a Potential Short-Term Memory.

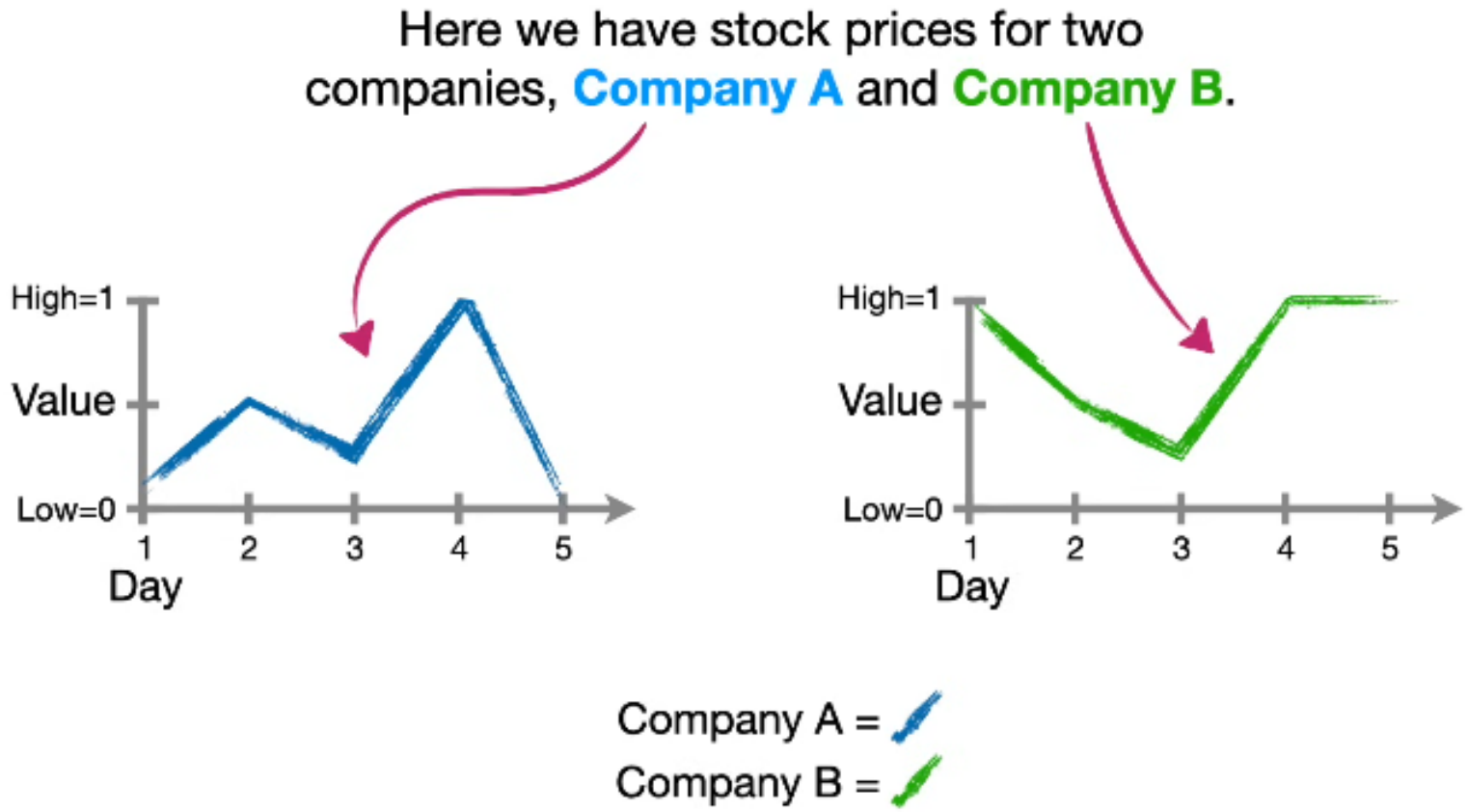

Long-Term Memory
2
×
1.99
New Long-Term Memory
sum
2.96
% Long-Term To Remember
0.997
% Potential Memory To Remember
0.97
Potential Long-Term Memory
Potential Short-Term Memory
0.99
1.0
0.97
+ 1.62
+ 0.62
+ -0.32
+ 0.59
sum
sum
sum
sum
x 2.70
x 2.00
x 1.41
x 4.38
x 1.63
x 1.65
x 0.94
1
Short-Term Memory
Input
1
Now the LSTM has to decide how much of this Potential Short-Term Memory to pass on…


Long-Term Memory
2
×
1.99
sum
New Long-Term Memory
2.96
% Long-Term To Remember
0.997
% Potential Memory To Remember
0.97
Potential Long-Term Memory
% Potential Memory To Remember
Potential Short-Term Memory
0.99
+ 1.62
sum
× 2.70
× 1.63
1.0
+ 0.62
sum
× 2.00
× 1.65
0.97
+ -0.32
sum
× 1.41
× 0.94
+ 0.59
sum
× 4.38
× -0.19
1
Short-Term Memory
Input
1
…and this is done using the exact same method we used two times earlier.

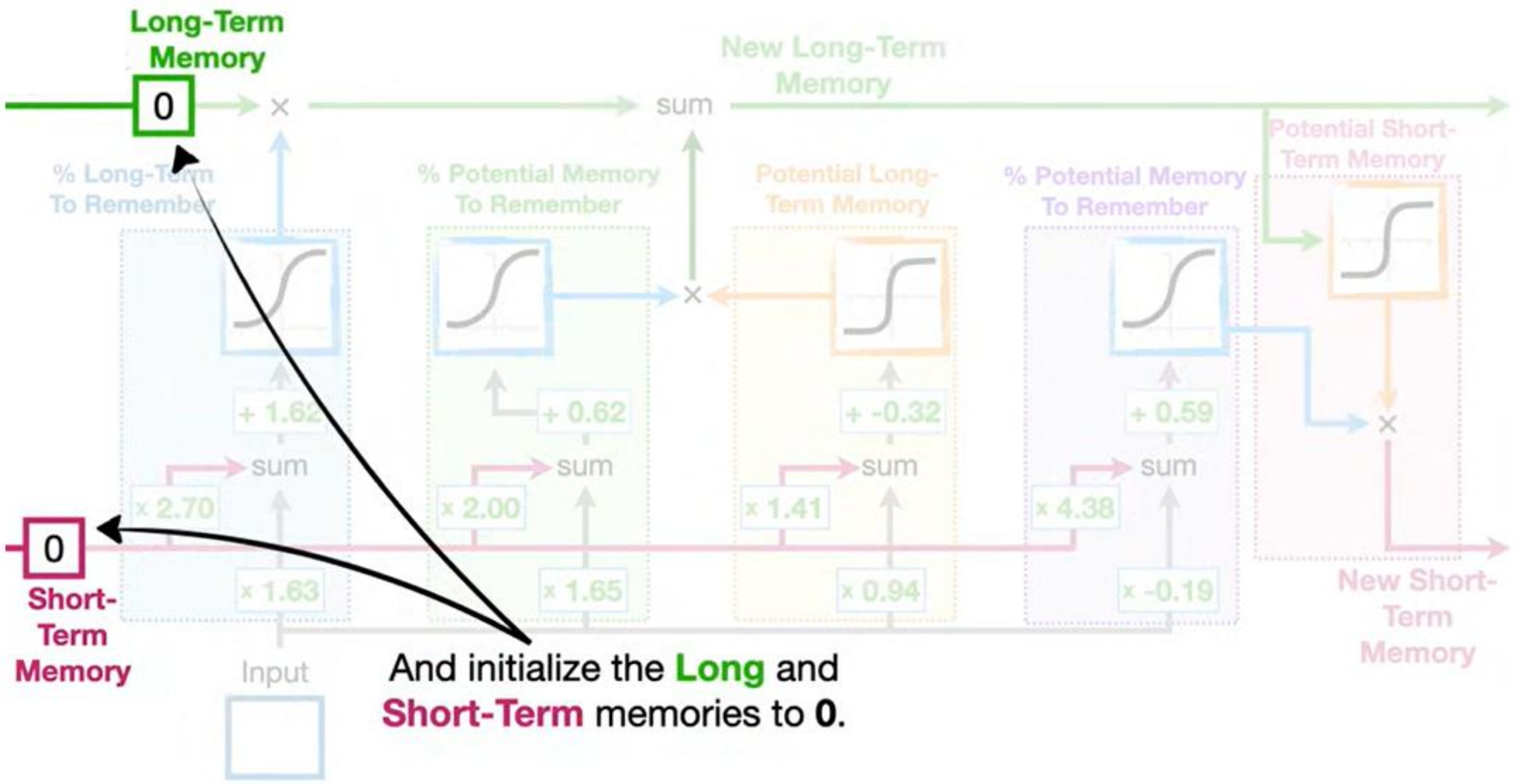


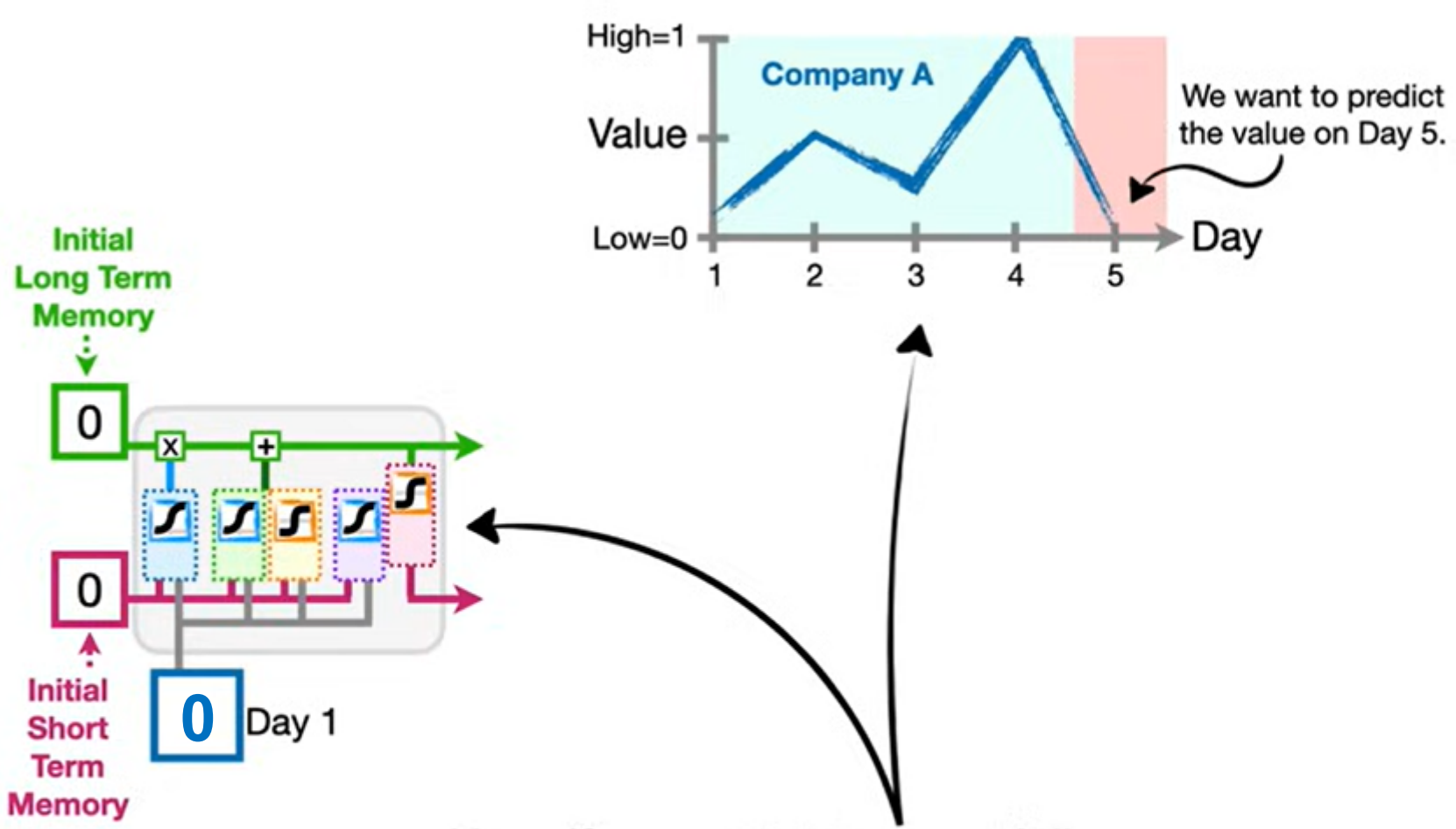
This part is called the **Output gate**

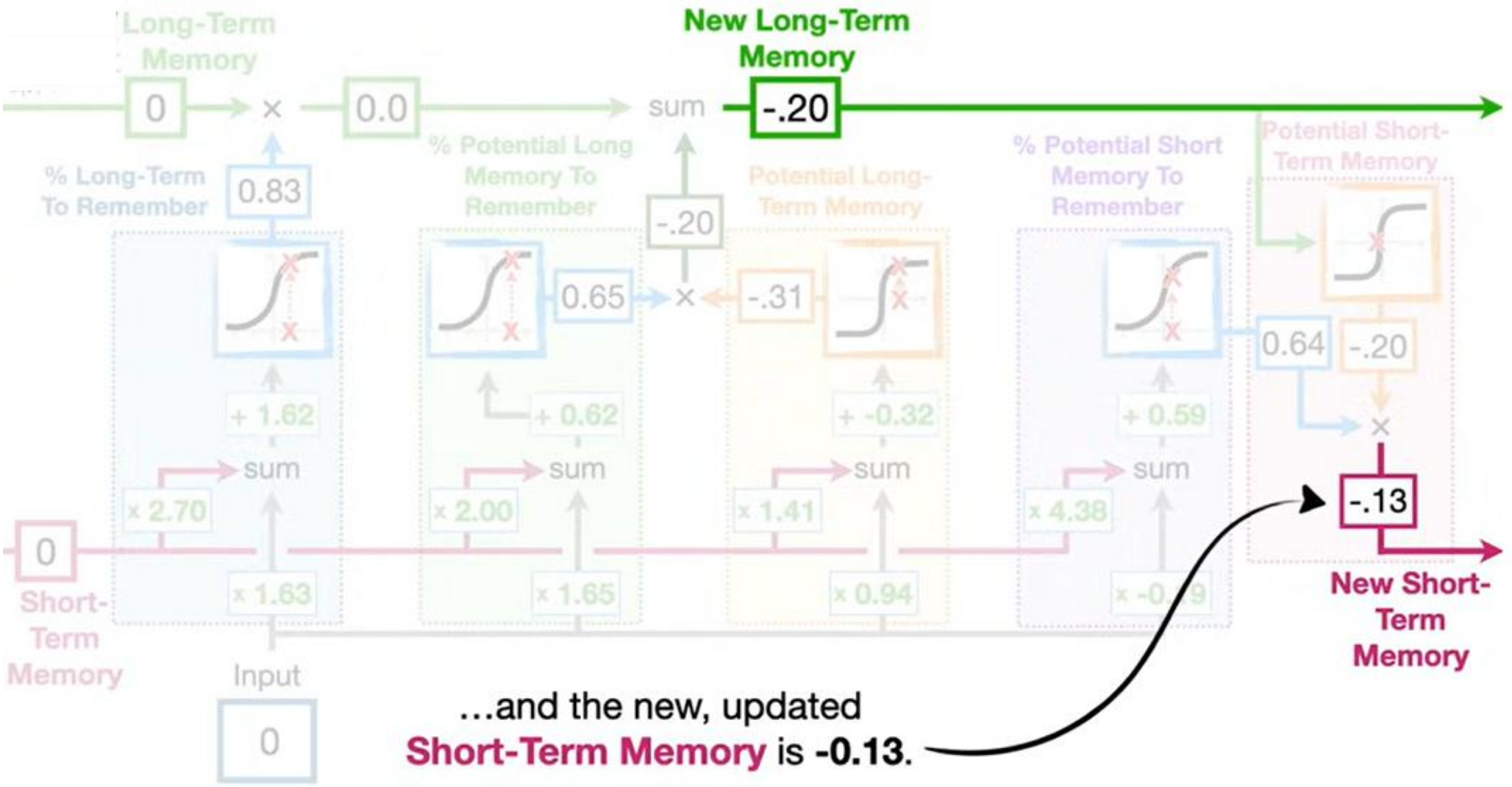
Let’s see an example

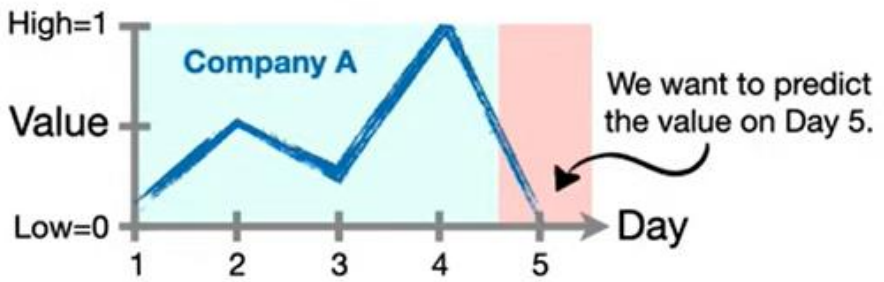


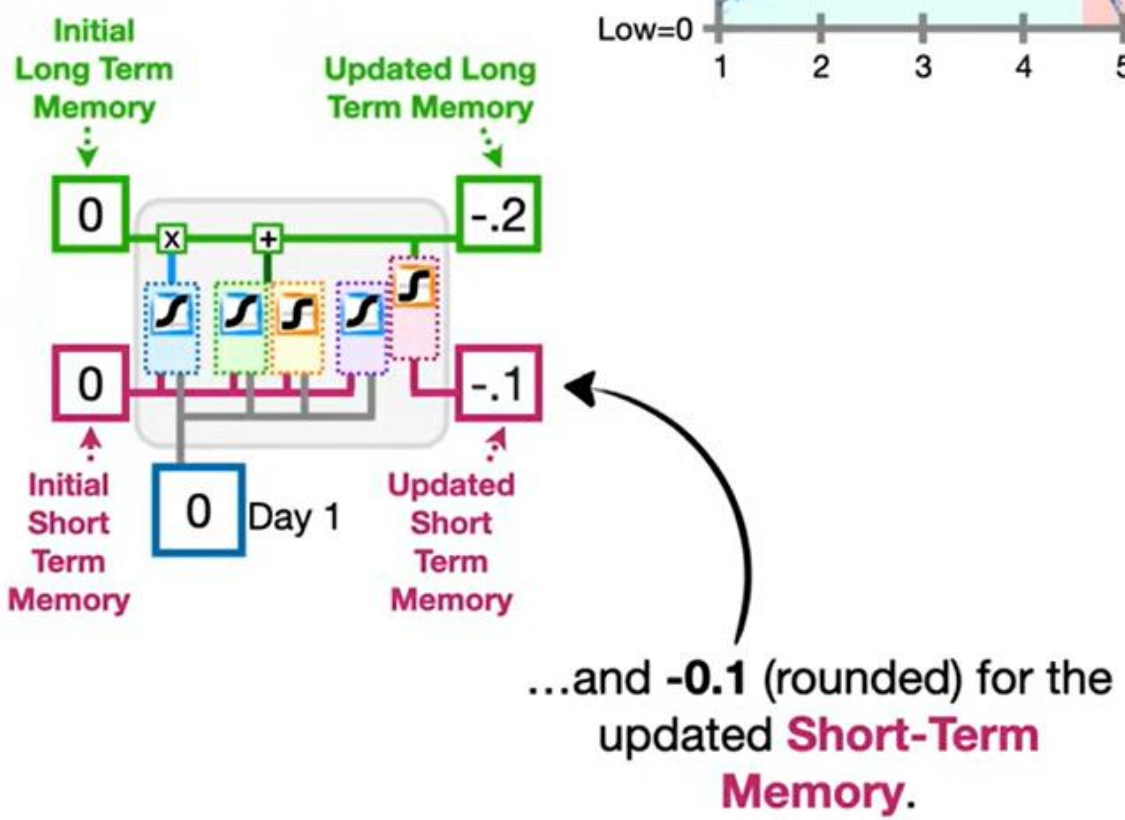
The only difference is at day 1 and day 5… we want the LSTM to remember what happened on day1 to predict what will happen in day 5

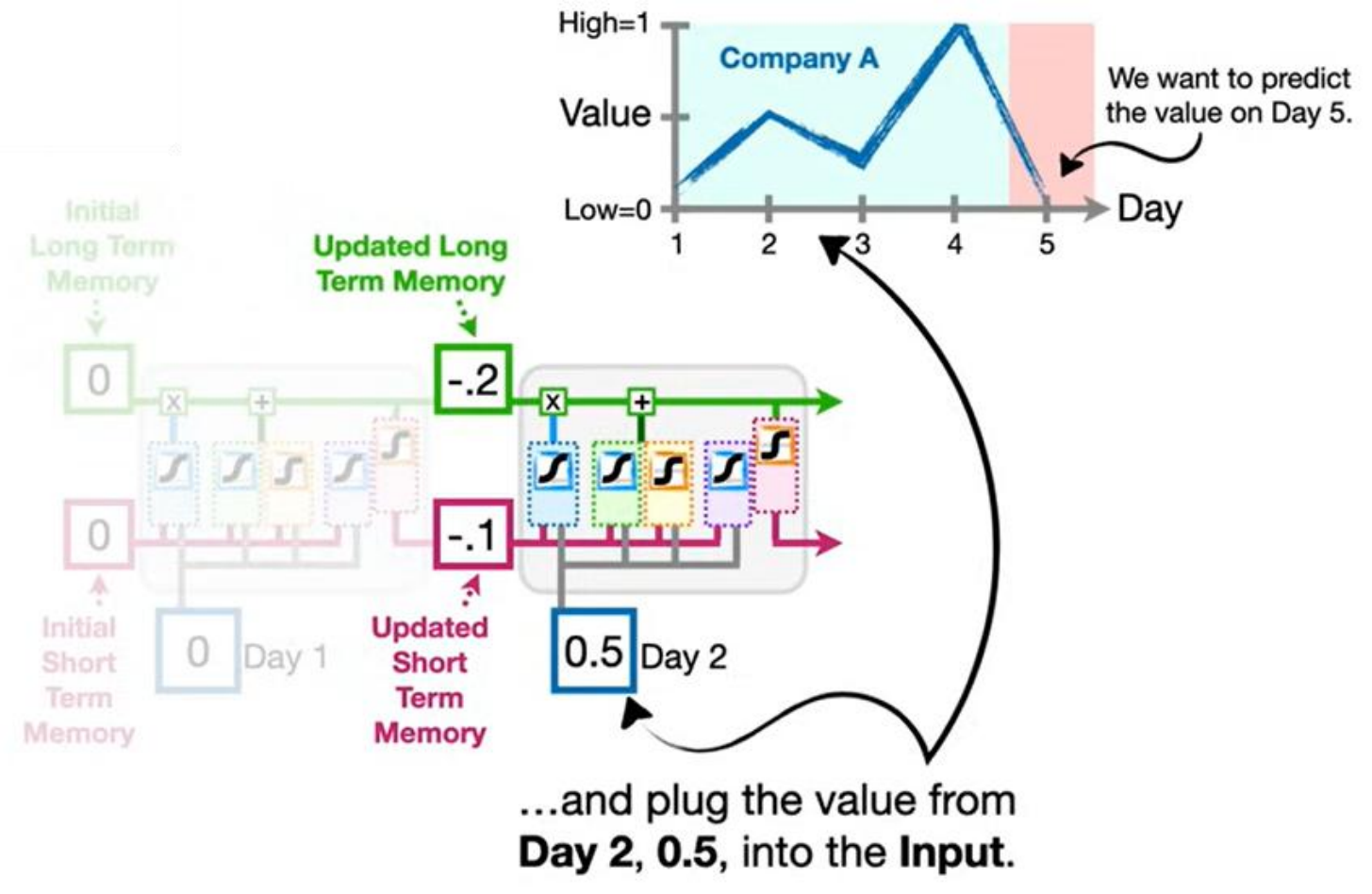

Long-Term Memory
0
New Long-Term Memory
sum
Potential Short-Term Memory
% Long-Term To Remember
% Potential Memory To Remember
Potential Long-Term Memory
% Potential Memory To Remember
+ 1.62
+ 0.62
+ -0.32
+ 0.59
sum
sum
sum
sum
x 2.70
x 2.00
x 1.41
x 4.38
0
Short-Term Memory
x 1.63
x 1.65
x 0.94
x -0.19
New Short-Term Memory
Input
And initialize the Long and Short-Term memories to 0.

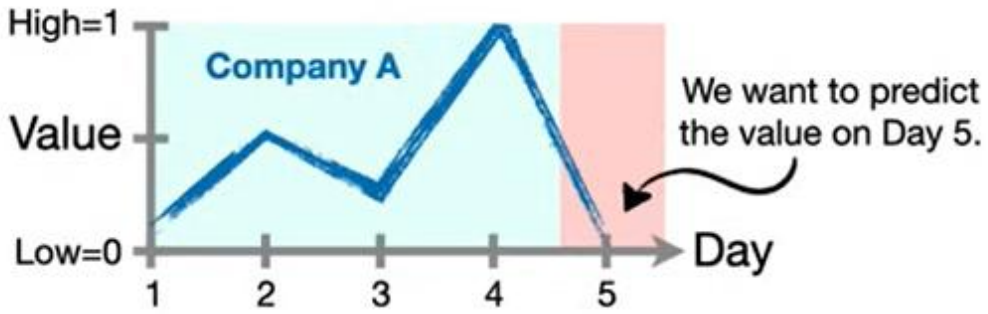

High=1
Value
Low=0
Company A
We want to predict the value on Day 5.
Day
1
2
3
4
5
Initial Long Term Memory
0
0
Initial Short Term Memory
0
Day 1
Now, if we want to sequentially run Company A's values from Days 1 through 4 through this LSTM…

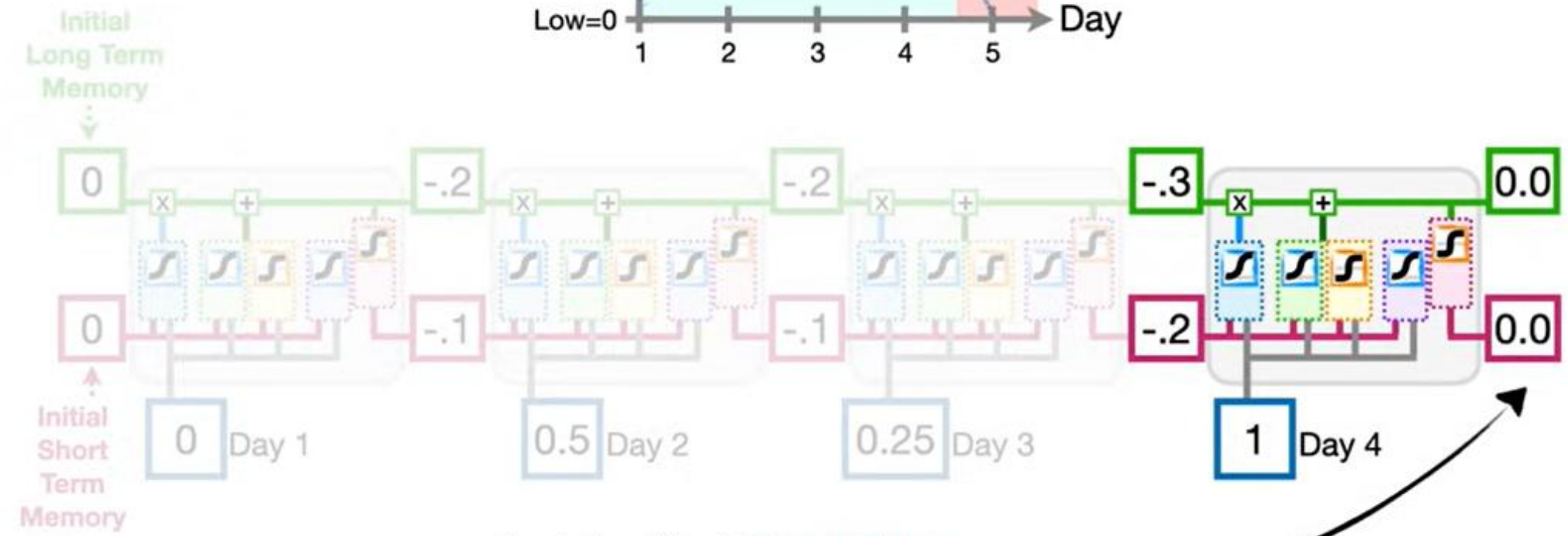

Long-Term Memory
0
×
0.0
sum
New Long-Term Memory
-.20
% Long-Term To Remember
0.83
% Potential Long Memory To Remember
-.20
Potential Long-Term Memory
% Potential Short Memory To Remember
Potential Short-Term Memory
0.65
×
-.31
0.64
-.20
+ 1.62
+ 0.62
+ -0.32
+ 0.59
sum
sum
sum
sum
× 2.70
× 2.00
× 1.41
× 4.38
×
-.13
0
Short-Term Memory
× 1.63
× 1.65
× 0.94
New Short-Term Memory
Input
0
…and the new, updated
Short-Term Memory is -0.13.


High=1
Value
Low=0
Company A
We want to predict the value on Day 5.
Day
1
2
3
4
5

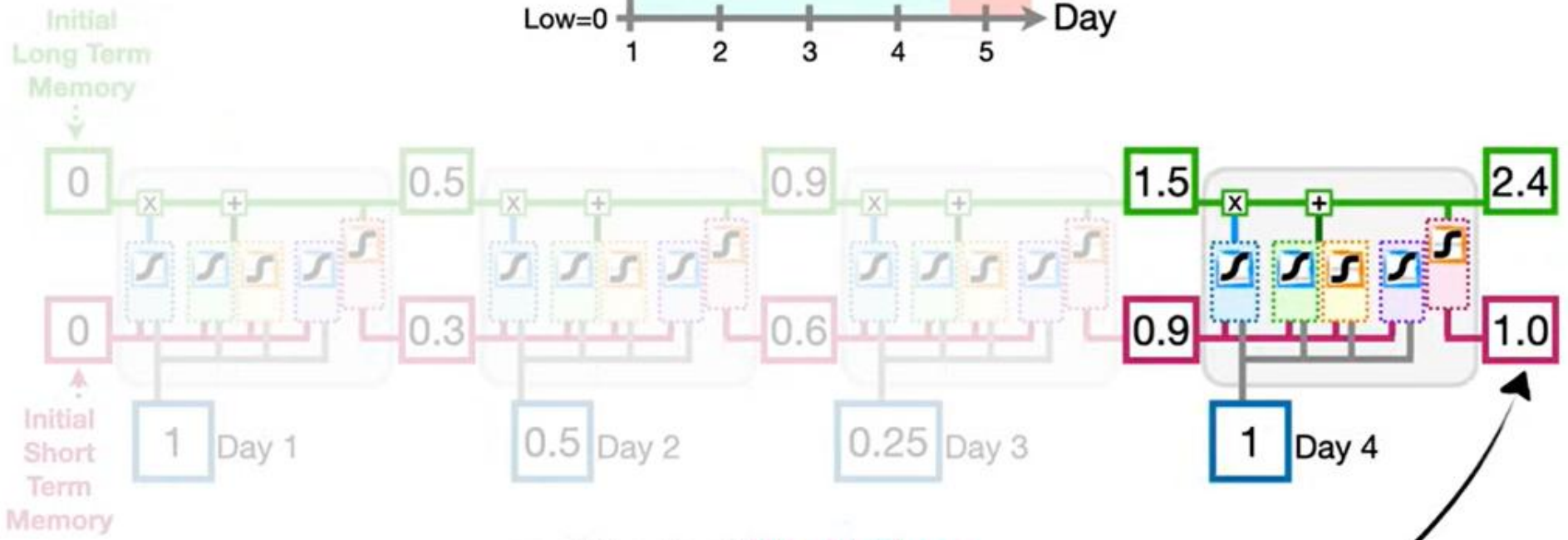

Initial Long Term Memory
Updated Long Term Memory
0
-.2
0
-.1
0
Day 1
Initial Short Term Memory
Updated Short Term Memory
…and -0.1 (rounded) for the updated Short-Term Memory.


High=1
Value
Low=0
Company A
We want to predict the value on Day 5.
Day
1
2
3
4
5
Initial Long Term Memory
Updated Long Term Memory
0
-.2
0
-.1
Initial Short Term Memory
0
Day 1
Updated Short Term Memory
0.5
Day 2
...and plug the value from Day 2, 0.5, into the Input.





And the final **Short-Term Memory**, **0.0**, is the **Output** from the unrolled **LSTM**.



…and the final **Short-Term Memory**, **1.0**, is the **Output** from the **unrolled LSTM**.